ZOVAK · NOFA HEALTH

How the AI receptionist works

Chat and voice booking, the booking card, moving and cancelling, reminders and call rules — step by step, with the real code.

Built October 30, 2026 from the current source of `zovak-backend` and `serena-health`.

Contents

1. The big picture
2. Running it
3. The safety rules
4. Booking states and the card
5. How the card is triggered: chat
6. How the card is triggered: voice
7. Booking a new appointment
8. Moving an appointment
9. Cancelling an appointment
10. Spoken confirmations
11. Reminder emails
12. Today's date and live booking status
13. Ending a call and guardrails
14. Phone calls (SIP, Telnyx)
15. The SMS booking form
16. WhatsApp chat and calls
17. Doctors and staff
18. Delay recovery and the staff portal
19. The widget UI
20. File map
21. Limits and what's next
22. Why it's built this way
23. Change log

1. The big picture

Two projects work together. **serena-health** is the Nofa Health website (Next.js); its chat and voice widget is the customer's screen. **zovak-backend** (Python, FastAPI) is ZOVAK, the receptionist's brain: the AI agent, the scheduler and Google Calendar.

Customer's browser
Nofa widget: chat bubbles, voice call, booking card

↓

Next.js routes (serena-health)
/api/agent/chat · /api/agent/session · /api/agent/booking — forward to ZOVAK on the server, so the browser never talks to ZOVAK directly

↓

ZOVAK FastAPI (zovak-backend)
Dev routes for chat, voice routes protected by a per-call token

↓

Agent + Scheduler
OpenAI Agents SDK receptionist proposes; the scheduler checks rules and commits

↓

Google Calendar · Resend · GPT-Live
Events with email invites · reminder emails · live voice

Voice audio does not go through ZOVAK: the browser streams audio straight to OpenAI's GPT-Live over WebRTC. ZOVAK attaches a *sideband* WebSocket to the same live session: it hears the caller's transcript, runs the agent when GPT-Live delegates a request, and tells GPT-Live what to say.

export async function zovakFetch · serena-health/src/lib/agent/zovak.ts

```
export async function zovakFetch(path: string, init?: RequestInit): Promise<Response> {
  const url = zovakUrl(path);
  if (!url) throw new ZovakUnavailable("ZOVAK_API_URL is not set");
  try {
    return await fetch(url, {
      ...init,
      headers: {
        "Content-Type": "application/json",
        // ngrok's free plan shows a "visit site" page to unknown callers; this skips it.
        "ngrok-skip-browser-warning": "1",
        ...(ZOVAK_SITE_KEY ? { "X-Zovak-Site-Key": ZOVAK_SITE_KEY } : {}),
        ...accessHeaders(),
        ...init?.headers,
      },
      cache: "no-store",
    });
  } catch (err) {
    throw new ZovakUnavailable(`Could not reach ZOVAK at ${url}: ${(err as Error).message}`);
  }
}
```

Every Next.js route uses `zovakFetch`. `ZOVAK_API_URL` in `.env.local` points at the backend (`http://127.0.0.1:8000` locally).

2. Running it

What	Command / setting
ZOVAK backend	`uvicorn zovak.main:app --reload --host 127.0.0.1 --port 8000` (conda env `Crewai`)
Website	`npm run dev` in serena-health, then open `http://localhost:3000`
Website → ZOVAK	`.env.local`: `ZOVAK_API_URL`, `NEXT_PUBLIC_AGENT_LIVE=true`
Google Calendar	Open `/api/v1/auth/google/start` after every backend restart (tokens live in memory)
Reminders	`.env`: `RESEND_API_KEY`, `REMINDER_FROM_EMAIL` (verified domain)
Tests	`python -m pytest -q` in zovak-backend; `npx tsc --noEmit` in serena-health
Deploy (Vercel + ngrok)	zovak-backend/.env: PUBLIC_BASE_URL=https://stapling-delete-stunner.ngrok-free.dev and SITE_API_KEY=<random>; run ngrok on port 8000 with that domain. Vercel env: ZOVAK_API_URL (same ngrok URL, or edit src/config/backend.ts), ZOVAK_SITE_KEY (same random value), DEMO_ACCESS_CODE, NEXT_PUBLIC_AGENT_LIVE=true

On start-up ZOVAK creates the Google-backed scheduler and, if Resend is configured, starts the reminder job:

lifespan · zovak/main.py

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    from zovak.providers.notifications.ntfy_push import NtfyPush
    from zovak.providers.notifications.sms_router import SmsRouter
    from zovak.providers.notifications.telnyx_sms import TelnyxSms
    from zovak.services.booking_forms import BookingForms
    from zovak.services.google_scheduler import GoogleBackedScheduler
    from zovak.workers.reminders import ReminderWorker
    # Load the AI library now (~2.5 s), not during the first patient's first message.
    import agents  # noqa: F401
    from agents.extensions.memory import AsyncSQLiteSession  # noqa: F401
    import zovak.agents.receptionist  # noqa: F401
    app.state.scheduler = GoogleBackedScheduler(app)
    settings = app.state.settings = get_settings()
    if settings.demo_seed:
        from zovak.services.clinic import seed_demo_bookings
        logger.info("Loaded %d demo bookings for the demo doctors",
seed_demo_bookings(app.state.scheduler))
    app.state.booking_forms = BookingForms(app.state.scheduler,
minutes=settings.booking_form_minutes)
    app.state.sms = sms = SmsRouter()
    if settings.test_sms_ntfy_topic:
        sms.test_push = NtfyPush(settings.test_sms_ntfy_topic, settings.ntfy_server)
    if settings.telnyx_api_key and settings.telnyx_from_number:
        skip = tuple(p.strip() for p in settings.telnyx_sms_skip_prefixes.split(",") if p.strip())
        sms.add_provider("telnyx", TelnyxSms(
            settings.telnyx_api_key, settings.telnyx_from_number,
            settings.telnyx_messaging_profile_id,
        ), skip=skip)
    if settings.whatsapp_access_token and settings.whatsapp_phone_number_id:
        from zovak.providers.notifications.whatsapp_meta import WhatsAppCloud
        from zovak.services.whatsapp_bot import WhatsAppBot
        cloud = WhatsAppCloud(settings.whatsapp_access_token, settings.whatsapp_phone_number_id,
                              graph_version=settings.whatsapp_graph_version)
        app.state.whatsapp = WhatsAppBot(app.state.scheduler, cloud,

max_messages_per_hour=settings.whatsapp_max_messages_per_hour)
        from zovak.services.whatsapp_calls import WhatsAppCalls
        app.state.whatsapp_calls = WhatsAppCalls(app.state, cloud)  # experiment
    from zovak.providers.notifications.resend_email import send_resend_email
    from zovak.services.recovery import RecoveryPlanner
    from zovak.services.staff_auth import StaffSessions

    async def email_sender(to, subject, page, text):
        if settings.resend_api_key and settings.reminder_from_email:
            await send_resend_email(settings.resend_api_key, settings.reminder_from_email, to,
                                    subject, page, text, idempotency_key=f"delay-{to}-{subject}-
{text[-40:]}")

    app.state.staff_sessions = StaffSessions()
    app.state.recovery = RecoveryPlanner(
        app.state.scheduler, sms=sms, email_sender=email_sender,
        whatsapp=getattr(getattr(app.state, "whatsapp", None), "cloud", None),
        public_base_url=settings.public_base_url or "http://127.0.0.1:8000",
    )
    # A Bangladeshi gateway plugs in later the same way (Telnyx already skips +880):
    # sms.add_provider("alpha-sms", AlphaSms(...), prefixes=("+880",))
    reminders = None
    if settings.resend_api_key and settings.reminder_from_email:
        worker = ReminderWorker(
            app.state.scheduler, api_key=settings.resend_api_key,
            sender=settings.reminder_from_email, minutes_before=settings.reminder_minutes_before,
```

```
        )
        reminders = asyncio.create_task(worker.run_forever(), name="zovak-reminders")
    else:
        logger.info("Appointment reminders off: set RESEND_API_KEY and REMINDER_FROM_EMAIL")
    yield
    if reminders is not None:
        reminders.cancel()
        await asyncio.gather(reminders, return_exceptions=True)
```

3. The safety rules

- **The AI proposes, the customer confirms.** Agent tools only create *pending* items (a booking proposal, a move, a cancellation). Nothing is booked, moved or cancelled until the customer taps a button or clearly says yes.
- **IDs come from the server.** Buttons send the ID of the single pending item; ZOVAK checks it matches what it holds for that conversation, so a stale or guessed ID is rejected.
- **Never say "booked" before the calendar confirms.** Messages are built from verified results, not the model's text.
- **Typed email, never spelled.** Saving an email never books anything.
- **Voice actions need the call's secret token** (`ui_token`), created fresh for every call.

4. Booking states and the card

Both chat and voice ask ZOVAK one question: *what should the card show now?* The answer is a `stage`, computed from the scheduler's records by one shared function.

Stage	Meaning	Card shows
`email_required`	Agent prepared an appointment, no email yet	Email box
`ready_to_confirm`	Pending appointment with email	Details + Confirm booking + Change email
`booked`	Confirmed booking in this conversation	Green card
`move_pending`	Agent proposed a new time	Old → new time + Confirm move / Keep
`cancel_pending`	Agent proposed cancelling	Cancel appointment / Keep it
`cancelled`	Booking was cancelled	Grey card
`idle`	Nothing yet	No card

booking_state · zovak/services/booking_state.py

```
async def booking_state(scheduler, tenant_id: str, conversation_id: str, *,
                        proposal: BookingProposal | None, email_ready: bool,
                        email_request_id: str | None) -> dict:
    """proposal/email_ready come from the channel, which owns its own email gate."""
    change = await scheduler.latest_change(tenant_id, conversation_id)
    booking = await scheduler.latest_booking(tenant_id, conversation_id)
    cancelled = scheduler.last_cancelled(tenant_id, conversation_id)

    if proposal is not None:
        stage = "ready_to_confirm" if email_ready else "email_required"
    elif change is not None:
        stage = "move_pending" if change.kind == "move" else "cancel_pending"
    elif booking is not None:
        stage = "booked"
    elif cancelled is not None:
        stage = "cancelled"
    else:
        stage = "idle"

    moving = getattr(scheduler, "moving_booking", lambda *_: None)(tenant_id, conversation_id)
    doctor_options, chosen = [], None
    if stage in ("idle", "booked", "cancelled") and proposal is None and change is None:
        doctor_options = await _doctor_options(scheduler, tenant_id, conversation_id)
        chosen_id = scheduler.chosen_doctor(tenant_id, conversation_id)
        if not doctor_options and chosen_id:
            from zovak.services.clinic import doctor_card
            chosen = doctor_card(scheduler.doctors[chosen_id])
            chosen["slots"] = await free_days(scheduler, tenant_id, conversation_id, chosen_id)
            picked = getattr(scheduler, "picked_time", lambda *_: None)(tenant_id, conversation_id)
            chosen["picked"] = picked.isoformat() if picked else None  # the card collapses to it
            chosen["purpose"] = "move" if moving else "book"  # new booking, or changing the booked
time
    return {
        "stage": stage,
        "doctor_options": doctor_options,
        "chosen_doctor": chosen,
        "proposal": _appointment(scheduler, proposal) if proposal else None,
        "booking": ({**_appointment(scheduler, booking),
                     # One time change per booking for patients ("Change time" on the booked card).
                     "can_change_time": booking.time_changes < MAX_TIME_CHANGES} if booking else
None),
        "change": _change(change) if change else None,
        "cancelled": _appointment(scheduler, cancelled) if cancelled and stage == "cancelled" else
None,
        "email_request_id": email_request_id if stage == "email_required" else None,
    }
```

The data behind it: a `BookingProposal` (pending or confirmed booking) and a `ChangeRequest` (pending move/cancel):

ChangeRequest · zovak/domain/schemas.py

```
class ChangeRequest(BaseModel):
    """A pending move or cancellation of a confirmed booking in the same conversation.

    Like a booking proposal, it changes nothing until the customer explicitly confirms it.
    """
    id: str
    tenant_id: str
    conversation_id: str
    booking_id: str
    kind: Literal["move", "cancel"]
    new_start: datetime | None = None
    new_end: datetime | None = None
    expires_at: datetime
    status: Literal["pending", "applied", "expired", "conflict"] = "pending"
```

5. How the card is triggered: chat

1. Customer types a message. The widget calls `sendChatMessage` → `/api/agent/chat` → ZOVAK `/api/v1/dev/chat`.
2. ZOVAK runs the receptionist agent. If details are complete it calls the `propose_booking` tool, which stores a *pending* proposal.
3. The reply streams into the chat word by word.
4. Right after the reply, the widget asks for the card: `fetchBookingState(getChatChannel())` → `/api/agent/booking?channel=chat` → ZOVAK `/api/v1/dev/booking-state`.
5. ZOVAK answers with the stage (e.g. `ready_to_confirm`) and the widget renders `BookingPanel` under the messages.
6. A button tap calls `sendBookingAction`, then fetches the state again so the card updates instantly.

ChatView.send · serena-health/src/components/assistant/AssistantWidget.tsx

```
  const send = useCallback(
    async (text: string) => {
      const content = text.trim();
      if (!content || busy) return;
      const next = [...messages, { id: nextId(), role: "user" as const, content }];
      setMessages(next);
      setInput("");
      setPending(true);
      try {
        if (AGENT_DEMO) {
          const reply = await sendChatMessage(next);
          setPending(false);
          await streamReply(reply);
          return;
        }
        // Nofa's words appear as the AI writes them.
        const id = nextId();
        let started = false;
        const full = await streamChatMessage(next, (soFar) => {
          if (!started) {
            started = true;
            setPending(false);
            setStreamingId(id);
            setMessages((prev) => [...prev, { id, role: "assistant", content: soFar }]);
          } else {
            setMessages((prev) => prev.map((m) => (m.id === id ? { ...m, content: soFar } : m)));
          }
        });
        const card = await fetchBookingState(getChatChannel());
        if (!started) setMessages((prev) => [...prev, { id, role: "assistant", content: full }]);
        // Same tick as the reply finishing: no old card flashes up; the card only shows
        // when it changed or the message was about the booking.
        const key = cardKeyOf(card);
        setShowCard(key !== lastCardKey.current || ABOUT_BOOKING.test(content));
        lastCardKey.current = key;
        setBooking(card);
        setStreamingId(null);
      } catch {
        setMessages((prev) => [
          ...prev,
          { id: nextId(), role: "assistant", content: "Sorry — I couldn't reach the scheduling
system. Please try again in a moment." },
        ]);
      } finally {
        setPending(false);
        setStreamingId(null);
      }
    },
    [messages, busy]
  );
```

export async function fetchBookingState · serena-health/src/lib/agent/client.ts

```
export async function fetchBookingState(ch: BookingChannel): Promise<BookingState | null> {
  const params = new URLSearchParams({ channel: ch.channel, conversationId: ch.conversationId });
  try {
    const res = await fetch(`/api/agent/booking?${params}`, {
      headers: ch.token ? { "x-voice-token": ch.token } : {},
      cache: "no-store",
    });
    if (!res.ok) {
      console.warn("[booking] state check failed", res.status, await res.text());
      return null;
    }
    return (await res.json()) as BookingState;
  } catch {
    return null;
  }
}
```

chat_booking_state · zovak/api/routes/dev.py

```
@router.get("/booking-state")
async def chat_booking_state(conversation_id: str, request: Request,
                             tenant_id: str = "demo-business") -> dict:
    scheduler = request.app.state.scheduler
    try:
        pending = await scheduler.latest_pending(tenant_id, conversation_id)
    except SchedulingError as exc:
        raise HTTPException(status_code=404, detail=str(exc)) from exc
    return await booking_state(
        scheduler, tenant_id, conversation_id, proposal=pending,
        email_ready=bool(pending and pending.customer_email),
        email_request_id=pending.id if pending else None,
    )
```

6. How the card is triggered: voice

1. Customer taps Start call. The browser makes a WebRTC offer and sends it via `/api/agent/session` to ZOVAK `/api/v1/dev/voice/session`.
2. ZOVAK creates the GPT-Live session (with today's date in the instructions), attaches its sideband and returns the answer, the `conversation_id` and a fresh `ui_token`.
3. Audio flows browser ↔ OpenAI. When the caller asks for something, GPT-Live *delegates*; the sideband runs `_handle_turn`, which runs the agent and returns what Nofa should say.
4. Meanwhile the browser polls the card every 1.5 seconds: `pollBooking` → `/api/agent/booking?channel=voice` (with the token) → ZOVAK `/voice/booking-state`.
5. As soon as a proposal, move or cancel exists, the card appears in the voice panel. A tap runs the action, and the sideband also *speaks* the verified result on the call.

startLiveVoiceSession → pollBooking · serena-health/src/lib/agent/client.ts

```
  const pollBooking = async () => {
    if (stopped || !call) return;
    const state = await fetchBookingState(call);
    if (state?.callEnded) return endedRemotely(state.callEnded);
    // Nofa is saying goodbye: mute the mic so nothing more is heard.
    if (state?.ending) mic?.getAudioTracks().forEach((t) => (t.enabled = false));
    if (state && !stopped) {
      const key = JSON.stringify(state);
      if (key !== lastState) {
        lastState = key;
        handlers.onBookingState?.(state, call);
      }
    }
    if (!stopped) pollTimer = setTimeout(pollBooking, 1500);
  };
```

VoiceSideband.booking_state · zovak/voice/sideband.py

```
async def booking_state(self) -> dict:
    """What the voice panel should show; the website polls this during the call."""
    async with self._agent_lock:
        pending = self._armed_proposal(
            await self.scheduler.latest_pending(_TENANT, self.conversation_id)
        )
        if pending is not None and self.email_form.proposal_id != pending.id:
            self._arm_email(pending)
        email_ready = bool(
            pending and self.email_form.verified
            and self.email_form.verified_email == pending.customer_email
        )
        state = await booking_state(
            self.scheduler, _TENANT, self.conversation_id, proposal=pending,
            email_ready=email_ready, email_request_id=self.email_form.request_id,
        )
        return {**state, "call_ended": self.call_ended, "ending": self.guard.ending}
```

_get_authorized_voice · zovak/api/routes/voice_dev.py

```
def _get_authorized_voice(request: Request, conversation_id: str, authorization: str | None):
    # LOCAL DEV ONLY. For production use signed-in users and tenant/session auth.
    handlers = getattr(request.app.state, "voice_sidebands", {})
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, detail="Voice session token required")
    token = authorization[7:]
    for handler in handlers.values():
        if (handler.conversation_id == conversation_id and not handler.closed
                and compare_digest(handler.ui_token, token)):
            return handler
    raise HTTPException(403, detail="Invalid or expired voice session")
```

6.1 Under the hood: how a website call connects to GPT-Live (WebRTC)

Three separate connections make up one call. Only the first carries sound:

Connection	Between	Carries
WebRTC audio + a small data channel	Browser ↔ OpenAI	Your voice, Nofa's voice; events like `session.started` and live transcripts
HTTPS (one request)	Browser → Next.js → ZOVAK → OpenAI	The WebRTC offer/answer that sets up the audio
Sideband WebSocket	ZOVAK ↔ OpenAI	Control only: transcripts in, delegated requests in, Nofa's verified replies out

1. Start call: the browser opens the microphone, creates an `RTCPeerConnection` and a data channel, and writes an **SDP offer** (a description of how it can send and receive audio).
2. The offer goes to `/api/agent/session` (Next.js) → ZOVAK `/api/v1/dev/voice/session`. ZOVAK holds the OpenAI key, so the browser never sees it.
3. ZOVAK calls `client.live.create(...)` with the offer, Nofa's instructions (plus today's date) and `delegation: client`. OpenAI returns an **SDP answer** and a session ID.
4. ZOVAK attaches its **sideband** to that session (`wss://api.openai.com/v1/live/sessions/{id}/attach`), then returns the answer, a conversation ID and a per-call `ui_token`.
5. The browser applies the answer: audio now flows directly browser ↔ OpenAI. OpenAI sends `session.started` on the data channel → the widget switches to "Listening…".
6. When you ask for something, GPT-Live **delegates**: the sideband receives `session.delegation.created`, ZOVAK runs the agent and booking rules, and sends the reply back as `session.commentary.append`; Nofa says it.

function startLiveVoiceSession(· serena-health/src/lib/agent/client.ts

```
function startLiveVoiceSession(handlers: VoiceSessionHandlers): VoiceSession {
  let stopped = false;
  let pc: RTCPeerConnection | null = null;
  let mic: MediaStream | null = null;
  let channel: RTCDataChannel | null = null;
  let call: BookingChannel | null = null;
  let lastState = "";
  let pollTimer: ReturnType<typeof setTimeout> | null = null;
  const audio = new Audio();
  audio.autoplay = true;

  // Transcript deltas arrive word by word. Stream them into one entry per
  // speaker turn; a new turn starts when the other side starts talking.
  let turn = 0;
  // UI updates are batched to one per animation frame so a fast stream of
  // words doesn't re-render the panel dozens of times a second.
  let buffer: { id: string; role: "user" | "assistant"; text: string } | null = null;
  let frame: number | null = null;
  const emit = () => {
    frame = null;
    if (buffer?.text.trim()) handlers.onTranscript({ id: buffer.id, role: buffer.role, text:
buffer.text.trim() });
  };
  const flush = () => {
    if (frame !== null) cancelAnimationFrame(frame);
    emit();
    buffer = null;
  };
  const append = (role: "user" | "assistant", delta: string) => {
    if (buffer && buffer.role !== role) flush();
    buffer ??= { id: `turn-${++turn}`, role, text: "" };
    buffer.text += delta;
    frame ??= requestAnimationFrame(emit);
  };

  // The backend said goodbye (silence or off-topic) or GPT-Live closed: hang up.
  const endedRemotely = (reason: CallEndReason) => {
    if (stopped) return;
    stopped = true;
    if (channel?.readyState === "open") channel.send(JSON.stringify({ type: "session.close" }));
    cleanup();
    handlers.onCallEnded?.(reason);
    handlers.onStateChange("idle");
  };

  // Keep the booking card in step with ZOVAK while the call is live.
  const pollBooking = async () => {
    if (stopped || !call) return;
    const state = await fetchBookingState(call);
    if (state?.callEnded) return endedRemotely(state.callEnded);
    // Nofa is saying goodbye: mute the mic so nothing more is heard.
    if (state?.ending) mic?.getAudioTracks().forEach((t) => (t.enabled = false));
    if (state && !stopped) {
      const key = JSON.stringify(state);
      if (key !== lastState) {
        lastState = key;
        handlers.onBookingState?.(state, call);
      }
    }
    if (!stopped) pollTimer = setTimeout(pollBooking, 1500);
  };
```

```
  const cleanup = () => {
    flush();
    if (pollTimer) clearTimeout(pollTimer);
    pollTimer = null;
    channel?.close();
    pc?.close();
    mic?.getTracks().forEach((t) => t.stop());
    audio.srcObject = null;
    pc = null;
    mic = null;
    channel = null;
  };

  const fail = (message: string) => {
    if (stopped) return;
    stopped = true;
    cleanup();
    handlers.onError?.(message);
    handlers.onStateChange("idle");
  };

  handlers.onStateChange("connecting");

  (async () => {
    try {
      pc = new RTCPeerConnection();
      pc.addEventListener("track", (e) => {
        audio.srcObject = new MediaStream([e.track]);
        audio.play().catch(() => {});
      });
      // Echo cancellation matters most: without it Nofa hears its own voice
      // through the speakers, treats it as the caller talking, and cuts itself off.
      mic = await navigator.mediaDevices.getUserMedia({
        audio: { echoCancellation: true, noiseSuppression: true, autoGainControl: true },
… 84 more lines in the file
```

create_voice_session · zovak/api/routes/voice_dev.py

```
@router.post("/session", status_code=201)
async def create_voice_session(body: SDPOffer, request: Request) -> dict:
    settings = request.app.state.settings
    if not settings.openai_api_key:
        raise HTTPException(status_code=503, detail="Set OPENAI_API_KEY in .env")
    # This URL is development-only and is registered only when ENABLE_DEV_ROUTES=true.
    try:
        async with AsyncOpenAI(api_key=settings.openai_api_key, max_retries=0) as client:
            result = await client.live.create(
                session={
                    "model": "gpt-live-1",
                    "instructions": LIVE_PROMPT + " " + today_instructions() + " This is a local
scheduling DEMO. Only say a booking is confirmed if verified by the backend. Avoid speaking
proposal IDs aloud; let the screen display them.",
                    "delegation": {"type": "client"},
                },
                transport={"type": "webrtc", "sdp": body.sdp},
            )
    except (APIError, AttributeError) as exc:
        logger.exception("Could not create GPT-Live session")
        raise HTTPException(
            status_code=getattr(exc, "status_code", None) or 502,
            detail="GPT-Live session failed. Check API model access, billing and SDK version.",
        ) from exc

    conversation_id = f"voice-{uuid4().hex}"
    session_id = result.session.id
    if not hasattr(request.app.state, "voice_sidebands"):
        request.app.state.voice_sidebands = {}
    active: dict = request.app.state.voice_sidebands
    handler = VoiceSideband(
        session_id=session_id,
        conversation_id=conversation_id,
        api_key=settings.openai_api_key,
        scheduler=request.app.state.scheduler,
        agent_model=settings.openai_agent_model,
        on_close=lambda sid: active.pop(sid, None),
    )
    active[session_id] = handler
    handler.start()
    try:
        await asyncio.wait_for(handler.ready.wait(), timeout=10)
    except asyncio.TimeoutError as exc:
        await handler.stop()
        raise HTTPException(status_code=502, detail="GPT-Live sideband did not connect") from exc
    if handler.failed or handler.closed:
        await handler.stop()
        raise HTTPException(status_code=502, detail="GPT-Live sideband could not attach; check
server logs")

    return {
        "session": {"id": session_id},
        "transport": {"type": "webrtc", "sdp": result.transport.sdp},
        "conversation_id": conversation_id,
        "ui_token": handler.ui_token,
    }
```

VoiceSideband._listen · zovak/voice/sideband.py

```
async def _listen(self) -> None:
    url = f"wss://api.openai.com/v1/live/sessions/{self.session_id}/attach"
    try:
        async with connect(
            url,
            additional_headers={"Authorization": f"Bearer {self.api_key}"},
            max_size=8 * 1024 * 1024,
            ping_interval=20,
            open_timeout=10,
        ) as socket:
            self._socket = socket
            self._connected_at = monotonic()
            self.ready.set()
            self._watchdog = asyncio.create_task(self._watch_silence())
            logger.info("GPT-Live sideband connected for %s call %s",
                        self.channel, self.conversation_id)
            if self.channel == "phone":
                await self._speak(_GREETING)  # if you hear this, audio to the phone works
            async for raw in socket:
                event = json.loads(raw)
                kind = event.get("type", "")
                if self.guard.ending and kind in (
                    "session.input_transcript.delta", "session.delegation.created",
                ):
                    continue  # Nofa said goodbye: nothing the caller says counts now
                if kind == "session.input_transcript.delta":
                    if not self._heard_caller:
                        self._heard_caller = True
                        logger.info("Caller audio is arriving (%.1f s after connect)",
                                    monotonic() - self._connected_at)
                    self.transcripts.ingest(event)
                    self._caller_said(str(event.get("delta") or ""))
                    self._watch_for_goodbye()
                elif kind == "session.output_transcript.delta":
                    if not self._nofa_spoke:
                        self._nofa_spoke = True
                        logger.info("Nofa is speaking (%.1f s after connect)",
                                    monotonic() - self._connected_at)
                    self.guard.agent_speaking(monotonic(), 1.5)
                elif kind == "session.delegation.created":
                    delegation = event.get("delegation") or {}
                    delegation_id = delegation.get("id")
                    if delegation.get("target") != "client" or not delegation_id:
                        continue
                    if delegation_id in self._delegations:
                        continue
                    self._delegations.add(delegation_id)
                    task = asyncio.create_task(self._delegate(socket, delegation_id,
int(event.get("offset_ms", 0))))
                    self._pending.add(task)
                    task.add_done_callback(self._pending.discard)
                elif kind == "session.closed":
                    logger.info("GPT-Live session ended: %s", self.session_id)
                    return
                elif kind == "error" or kind == "session.error":
                    logger.warning("GPT-Live session error: %s", str(event.get("error", event))
[:400])
    except asyncio.CancelledError:
        raise
    except Exception as exc:
        self.failed = str(exc)
        logger.exception("GPT-Live sideband connection failed")
```

```
    finally:
        if self._watchdog is not None:
            self._watchdog.cancel()
        self._socket = None
        self.ready.set()
        self.closed = True
        if self._on_close:
            self._on_close(self.session_id)
```

VoiceSideband._delegate · zovak/voice/sideband.py

```
async def _delegate(self, socket, delegation_id: str, offset_ms: int) -> None:
    # A short grace period for transcript fragments; this is NOT a
    # production-grade finished-turn signal.
    await asyncio.sleep(0.6)
    text = self.transcripts.take_for_delegation(offset_ms)
    self._turn_heard_at = monotonic()
    logger.info("Caller said: %r", text[:160])
    if text and not is_meaningful(text):
        reply = self._noise_only_turn()
    elif not text:
        reply = "I didn't catch the full request. Please repeat it."
    else:
        async with self._agent_lock:
            try:
                reply = await self._handle_turn(text, self._run_agent)
                logger.info("Nofa replies: %r", reply[:160])
            except Exception:
                logger.exception("Voice delegated agent or confirmation failed")
                reply = "Sorry, I couldn't complete that request. Please try again."
    if not self.closed:
        self.guard.agent_speaking(monotonic(), speech_seconds(reply))
        try:
            await socket.send(json.dumps({
                "type": "session.commentary.append",
                "event_id": f"zovak_{uuid4().hex}",
                "delegation_id": delegation_id,
                "content": reply,
            }))
        except Exception:
            logger.exception("Could not return GPT-Live delegation result")
```

7. Booking a new appointment

7.1 The agent proposes

propose_booking · zovak/agents/receptionist.py

```
@function_tool
async def propose_booking(
    ctx: RunContextWrapper[AgentContext], service_id: str,
    start_iso: str, customer_name: str, customer_email: str | None = None,
    doctor_id: str | None = None,
) -> str:
    """Prepare a pending booking proposal ONLY. Does not book anything.

    start_iso must include a timezone offset, e.g. 2026-11-01T14:00:00+06:00.
    """
    try:
        # A spoken email is untrusted in WebRTC voice mode. The browser
        # submits the exact typed value through a separate backend route.
        if ctx.context.email_via_form:
            customer_email = None
        proposal = await ctx.context.scheduler.propose(
            ctx.context.tenant_id,
            ctx.context.conversation_id,
            service_id,
            datetime.fromisoformat(start_iso),
            customer_name,
            customer_email=customer_email,
            doctor_id=doctor_id,
            customer_phone=ctx.context.customer_phone,
        )
        print(
            f"[TOOL EXECUTED] propose_booking | "
            f"proposal_id={proposal.id} | "
            f"service={proposal.service_id} | "
            f"start={proposal.start.isoformat()}",
            flush=True
        )

        return (
            f"PENDING proposal_id={proposal.id}; "
            f"service={proposal.service_id}; "
            f"doctor={ctx.context.scheduler.doctor_name(proposal.doctor_id)}; "
            f"start={proposal.start.isoformat()}; customer_email={proposal.customer_email}. "
            "Tell customer this is NOT confirmed. "
            "Customer must explicitly confirm this proposal through the confirmation action."
        )
    except (ValueError, SchedulingError) as exc:
        return f"Cannot propose booking: {exc}"
```

In voice calls `email_via_form` is on, so any spoken email is dropped: the email must be typed in the box. The scheduler checks the slot is really free and keeps only one pending proposal per conversation:

DemoScheduler.propose · zovak/services/mock_scheduler.py

```
async def propose(
    self, tenant_id: str, conversation_id: str, service_id: str,
    start: datetime, customer_name: str, customer_email: str | None = None, *,
    doctor_id: str | None = None, customer_phone: str | None = None,
) -> BookingProposal:
    self._check_tenant(tenant_id)
    if service_id not in self.services:
        raise SchedulingError("Unknown service")
    if start.tzinfo is None:
        raise SchedulingError("An explicit timezone offset is required")
    local_start = start.astimezone(self.timezone)
    day_slots = await self.available_slots(
        tenant_id, service_id, local_start.date(), doctor_id=doctor_id,
        conversation_id=conversation_id,
    )
    slot = next((s for s in day_slots if s.start == local_start), None)
    if slot is None:
        raise SchedulingError("Slot unavailable or outside the doctor's hours")
    # A new time in the same conversation keeps the email the patient already typed,
    # even if the earlier proposal expired meanwhile.
    key = (tenant_id, conversation_id)
    if customer_email is not None:  # only a real address, never a note like "already entered"
        from zovak.voice.email_form import validate_typed_email
        try:
            customer_email = validate_typed_email(customer_email)
        except (SchedulingError, ValueError):
            customer_email = None
    if customer_email is None:
        customer_email = self._contact.get(key, (None, None))[1]
    self._remember_contact(key, customer_name, customer_email)
    proposed = BookingProposal(
        id=str(uuid4()), tenant_id=tenant_id, conversation_id=conversation_id,
        service_id=service_id, start=local_start,
        end=local_start + timedelta(minutes=self.services[service_id].duration_minutes),
        customer_name=customer_name, customer_email=customer_email,
        customer_phone=customer_phone, doctor_id=slot.doctor_id,
        expires_at=datetime.now(timezone.utc) + timedelta(minutes=5),
    )
    self._offers.pop((tenant_id, conversation_id), None)
    self._chosen.pop((tenant_id, conversation_id), None)
… 11 more lines in the file
```

7.2 The email box

The card shows the email box while the stage is `email_required`. Submitting goes through the shared action route; in voice it reaches `submit_email`, which validates the address, attaches it to the pending proposal and reads the details back — but never books.

VoiceSideband.submit_email · zovak/voice/sideband.py

```
async def submit_email(self, *, request_id: str, email: str) -> dict:
    """Backend-only typed email acceptance. NEVER confirms a booking."""
    self.guard.caller_active()
    if self.closed:
        raise SchedulingError("The voice session has ended")
    validated = validate_typed_email(email)
    async with self._agent_lock:
        pending = await self.scheduler.latest_pending("demo-business", self.conversation_id)
        gate = self.email_form
        if not pending or gate.proposal_id != pending.id or not gate.request_id:
            raise SchedulingError("No active email request for this call")
        if not compare_digest(gate.request_id, request_id):
            raise SchedulingError("The email request is stale or invalid")
        if gate.verified:
            if gate.verified_email != validated:
                raise SchedulingError("Email was already submitted. Ask ZOVAK to change it.")
        else:
            await self.scheduler.set_proposal_email(
                "demo-business", self.conversation_id, pending.id, validated
            )
            gate.verified_email = validated
            self._screen_email = validated
            await note_for_agent(_TENANT, self.conversation_id,
                                 "The customer entered their email on screen for the pending
appointment.")
        service = self.scheduler.services.get(pending.service_id)
        name = service.name if service else pending.service_id
        spoken_details = describe_appointment(pending, name).replace(
            f" Customer email: {pending.customer_email}.", " Email captured on screen."
        )
        message = (
            "Thanks, I received the email you entered on screen. "
            f"Please review these appointment details. {spoken_details} "
            "Nothing has been booked yet. Are you sure you'd like me to confirm it? "
            "If so, tap Confirm booking on your screen, or say 'Yes, confirm my appointment'."
        )
        # Spoken UI update is best-effort: acknowledgement is not proof of
        # actual audio playback. Keep a fallback for the next voice turn.
        self.needs_readback = True
        if self._socket is not None:
            try:
                await self._socket.send(json.dumps({
                    "type": "session.commentary.append",
                    "event_id": f"zovak_email_{uuid4().hex}",
                    "delegation_id": None,
                    "content": message,
… 7 more lines in the file
```

7.3 Every card button goes through one route

`/api/agent/booking` maps each button to the right ZOVAK endpoint for chat or voice:

const ACTIONS: Record<BookingAction · serena-health/src/app/api/agent/booking/route.ts

```
const ACTIONS: Record<BookingAction, Partial<Record<Channel, Target>>> = {
  email: {
    voice: (id, c, email) => ["/api/v1/dev/voice/email-input", { conversation_id: c, request_id:
id, email }],
    chat: (id, c, email) => ["/api/v1/dev/proposal-email", { conversation_id: c, proposal_id: id,
email }],
  },
  "reopen-email": {
    voice: (id, c) => ["/api/v1/dev/voice/reopen-email", { conversation_id: c, proposal_id: id }],
    chat: (id, c) => ["/api/v1/dev/proposal-email/clear", { conversation_id: c, proposal_id: id }],
  },
  "confirm-booking": {
    voice: (id, c) => ["/api/v1/dev/voice/confirm-booking", { conversation_id: c, proposal_id: id
}],
    chat: (id, c) => ["/api/v1/dev/confirm", { tenant_id: ZOVAK_TENANT, conversation_id: c,
proposal_id: id }],
  },
  "confirm-change": {
    voice: (id, c) => ["/api/v1/dev/voice/change/confirm", { conversation_id: c, change_id: id }],
    chat: (id, c) => ["/api/v1/dev/change/confirm", { conversation_id: c, change_id: id }],
  },
  keep: {
    voice: (id, c) => ["/api/v1/dev/voice/change/discard", { conversation_id: c, change_id: id }],
    chat: (id, c) => ["/api/v1/dev/change/discard", { conversation_id: c, change_id: id }],
  },
  "pick-slot": {
    voice: (start, c) => ["/api/v1/dev/voice/pick-slot", { conversation_id: c, start }],
  },
};
```

7.4 Confirm booking

VoiceSideband.confirm_booking · zovak/voice/sideband.py

```
async def confirm_booking(self, *, proposal_id: str) -> dict:
    """'Confirm booking' button. Same checks as a spoken approval."""
    self.guard.caller_active()
    if self.closed:
        raise SchedulingError("The voice session has ended")
    async with self._agent_lock:
        current = self._armed_proposal(
            await self.scheduler.latest_pending(_TENANT, self.conversation_id)
        )
        if current is None or not compare_digest(current.id, proposal_id):
            raise SchedulingError("This appointment is no longer waiting for confirmation")
        if (not self.email_form.verified
                or self.email_form.verified_email != current.customer_email):
            raise SchedulingError("Please enter your email first")
        message = await self._commit_booking(current)
    await self._speak(message)
    return {"message": message}
```

VoiceSideband._commit_booking · zovak/voice/sideband.py

```
async def _commit_booking(self, current) -> str:
    """Confirm the armed proposal. Caller holds the lock and has checked the email."""
    try:
        result = await self.scheduler.confirm(_TENANT, self.conversation_id, current.id)
    except SchedulingError as exc:
        still_pending = await self.scheduler.latest_pending(_TENANT, self.conversation_id)
        if still_pending is None or still_pending.id != current.id:
            self.confirmation.clear()
            self.email_form.clear()
        return f"I could not confirm that appointment: {exc}. Nothing was booked. Please try again
or choose another time."
    self.confirmation.clear()
    self.email_form.clear()
    self.wrap_up = True
    logger.info("Voice booking confirmed by backend: %s", result.booking_id)
    await note_for_agent(_TENANT, self.conversation_id,
                         "The customer confirmed the booking. The appointment is now CONFIRMED.")
    service = self.scheduler.services.get(current.service_id)
    name = service.name if service else current.service_id
    details = describe_appointment(current, name).replace(
        f" Customer email: {current.customer_email}.", " Email captured on screen."
… 13 more lines in the file
```

With Google connected, confirming creates the calendar event with a deterministic ID (so a retry after a network error can't create a duplicate) and asks Google to email the invitation:

GoogleBackedScheduler.confirm · zovak/services/google_scheduler.py

```
async def confirm(self, tenant_id: str, conversation_id: str, proposal_id: str) -> ConfirmReply:
    self._check_tenant(tenant_id)
    async with self._lock:  # Only local process. No cross-worker or Google atomicity.
        proposal = self._proposals.get(proposal_id)
        if (proposal is None or proposal.tenant_id != tenant_id
            or proposal.conversation_id != conversation_id):
            raise SchedulingError("Proposal not found for this conversation")
        if proposal.status == "confirmed":
            return ConfirmReply(booking_id=proposal.id, provider="google-calendar",
status="confirmed")
        if proposal.status != "pending":
            raise SchedulingError("Proposal is no longer pending")
        if datetime.now(timezone.utc) > proposal.expires_at:
            proposal.status = "expired"
            raise SchedulingError("Proposal expired; please start a new request")
        if not proposal.customer_email:
            raise SchedulingError("Customer email address is required")
        if self._overlaps(tenant_id, proposal.start, proposal.end, doctor_id=proposal.doctor_id):
            proposal.status = "conflict"
            raise SchedulingError("Slot was taken; choose a different time")
        cal = await self._google()
        try:
            event_id = google_event_id(proposal.id)
            already_created = await cal.invited_event_exists(
                event_id=event_id, start=proposal.start, end=proposal.end,
                customer_email=proposal.customer_email,
            )
            # A previous insertion may have succeeded but the HTTP response
            # was lost. In that case accept verified matching event.
            if not already_created:
                busy = self._external_busy(await cal.busy_intervals(proposal.start, proposal.end))
                if busy:
                    proposal.status = "conflict"
                    raise SchedulingError("Google Calendar is busy; choose a different time")
                service = self.services[proposal.service_id]
                await cal.create_invited_event(
                    event_id=event_id,
                    title=f"{service.name} — {proposal.customer_name}
({self.doctor_name(proposal.doctor_id)})",
                    start=proposal.start, end=proposal.end,
                    customer_email=proposal.customer_email,
                    description=f"Booked with ZOVAK (development). Booking reference:
{proposal.id}",
                )
        except (httpx.HTTPError, RuntimeError, ValueError) as exc:
            # Could have created the event before a network timeout. Retain
            # pending state so user can retry, with deterministic event ID.
            raise SchedulingError("Google could not verify event creation; retry or check
Calendar") from exc
        proposal.status = "confirmed"
        proposal.confirmed_at = datetime.now(timezone.utc)
        self._bookings[proposal.id] = proposal
… 2 more lines in the file
```

8. Moving an appointment

1. Customer: "Can we move it to 4:30?" The agent checks availability and calls `propose_move`.
2. The scheduler stores a pending `ChangeRequest(kind='move')`; the booking itself is untouched.

3. The card switches to `move_pending` (old time crossed out → new time). On a call, Nofa also asks: "Tap Confirm move, or say 'Yes, confirm the move.'"
4. Confirm move → `apply_change`: re-checks the slot, moves the Google event (PATCH, Google emails the update), then updates the booking and resets its reminder.

propose_move · zovak/agents/receptionist.py

```
@function_tool
async def propose_move(ctx: RunContextWrapper[AgentContext], new_start_iso: str) -> str:
    """Prepare moving THIS conversation's confirmed appointment to a new time. Moves nothing.

    new_start_iso must include a timezone offset, e.g. 2026-11-01T15:00:00+06:00.
    Check availability first. The customer must confirm the move separately.
    """
    try:
        change = await ctx.context.scheduler.propose_move(
            ctx.context.tenant_id, ctx.context.conversation_id,
            datetime.fromisoformat(new_start_iso),
        )
        return (
            f"PENDING move to start={change.new_start.isoformat()}. NOT moved yet. "
            "Customer must explicitly confirm the move through the confirmation action."
        )
    except (ValueError, SchedulingError) as exc:
        return f"Cannot propose move: {exc}"
```

DemoScheduler.propose_move · zovak/services/mock_scheduler.py

```
async def propose_move(self, tenant_id: str, conversation_id: str,
                       new_start: datetime) -> ChangeRequest:
    """Prepare a move ONLY. Nothing changes until apply_change()."""
    self._check_tenant(tenant_id)
    if new_start.tzinfo is None:
        raise SchedulingError("An explicit timezone offset is required")
    booking = await self._bookable_for_change(tenant_id, conversation_id)
    if booking.time_changes >= MAX_TIME_CHANGES:
        raise SchedulingError("The time of this appointment was already changed once. "
                              "For another change, please contact the clinic.")
    local_start = new_start.astimezone(self.timezone)
    if local_start == booking.start:
        raise SchedulingError("The appointment is already at that time")
    day_slots = await self.available_slots(
        tenant_id, booking.service_id, local_start.date(), doctor_id=booking.doctor_id,
        ignore_booking_id=booking.id, conversation_id=conversation_id,
    )
    if not any(slot.start == local_start for slot in day_slots):
        raise SchedulingError("That new time is unavailable or outside business hours")
    return await self._store_change(ChangeRequest(
        id=str(uuid4()), tenant_id=tenant_id, conversation_id=conversation_id,
        booking_id=booking.id, kind="move", new_start=local_start,
        new_end=local_start + (booking.end - booking.start),
        expires_at=datetime.now(timezone.utc) + timedelta(minutes=5),
    ))
```

DemoScheduler.apply_change · zovak/services/mock_scheduler.py

```
async def apply_change(self, tenant_id: str, conversation_id: str,
                       change_id: str) -> ChangeReply:
    self._check_tenant(tenant_id)
    async with self._lock:
        change = self._changes.get(change_id)
        if (change is None or change.tenant_id != tenant_id
                or change.conversation_id != conversation_id):
            raise SchedulingError("Change request not found for this conversation")
        booking = self._bookings.get(change.booking_id)
        done = "moved" if change.kind == "move" else "cancelled"
        if change.status == "applied":
            return ChangeReply(booking_id=change.booking_id, kind=change.kind,
                               provider=self.provider, status=done)
        if change.status != "pending":
            raise SchedulingError("This change request is no longer pending")
        if datetime.now(timezone.utc) > change.expires_at:
            change.status = "expired"
            raise SchedulingError("This change request expired; please ask again")
        if (booking is None or booking.status != "confirmed"
                or booking.conversation_id != conversation_id):
            raise SchedulingError("The appointment is no longer active")
        if change.kind == "move" and self._overlaps(
            tenant_id, change.new_start, change.new_end, ignore_booking_id=booking.id,
            doctor_id=booking.doctor_id,
        ):
            change.status = "conflict"
            raise SchedulingError("That new time was just taken; choose another time")
        await self._calendar_apply(change, booking)
        if change.kind == "move":
            booking.start, booking.end = change.new_start, change.new_end
            booking.confirmed_at = datetime.now(timezone.utc)
            booking.reminder_sent = False  # the reminder follows the new time
            booking.time_changes += 1  # patients get one change; staff moves don't count
        else:
            booking.status = "cancelled"
            del self._bookings[booking.id]
            self._cancelled[(tenant_id, conversation_id)] = booking
        for picks in (self._moving, self._chosen, self._picked):  # the change is done
            picks.pop((tenant_id, conversation_id), None)
        change.status = "applied"
        return ChangeReply(booking_id=booking.id, kind=change.kind,
                           provider=self.provider, status=done)
```

GoogleBackedScheduler._calendar_apply · zovak/services/google_scheduler.py

```
async def _calendar_apply(self, change: ChangeRequest, booking: BookingProposal) -> None:
    """Move or delete the booking's Google event before local state changes."""
    if booking.source == "demo":
        return  # demo bookings were never put in Google Calendar
    cal = await self._google()
    event_id = google_event_id(booking.id)
    try:
        if change.kind == "move":
            busy = self._external_busy(await cal.busy_intervals(change.new_start, change.new_end))
            if busy:
                change.status = "conflict"
                raise SchedulingError("Google Calendar is busy at the new time; choose another")
            await cal.move_event(event_id=event_id, start=change.new_start, end=change.new_end)
        else:
            await cal.cancel_event(event_id=event_id)
    except (httpx.HTTPError, RuntimeError, ValueError, KeyError) as exc:
        action = "move" if change.kind == "move" else "cancel"
        raise SchedulingError(
            f"Google could not verify the {action}; nothing was changed. Please retry"
        ) from exc
```

GoogleInvites.move_event · zovak/providers/calendar/google_invites.py

```
async def move_event(self, *, event_id: str, start: datetime, end: datetime) -> None:
    """Change an existing event's time; Google emails the attendee the update."""
    if start.tzinfo is None or end.tzinfo is None or start >= end:
        raise ValueError("Timezone-aware start < end required")
    async with httpx.AsyncClient(timeout=15) if self._client is None else _shared(self._client) as
client:
        response = await client.patch(
            self._url + "/events/" + quote(event_id, safe=""),
            params={"sendUpdates": "all"}, headers=self._headers(),
            json={"start": {"dateTime": start.isoformat()},
                  "end": {"dateTime": end.isoformat()}},
        )
        response.raise_for_status()
        moved = response.json()
        if (moved.get("id") != event_id or moved.get("status") == "cancelled"
                or datetime.fromisoformat(moved["start"]["dateTime"].replace("Z", "+00:00")) !=
start
                or datetime.fromisoformat(moved["end"]["dateTime"].replace("Z", "+00:00")) != end):
            raise RuntimeError("Google did not verify the event move")
```

When checking the new time, the booking's *own* slot counts as free (`ignore_booking_id`), otherwise a small shift like 2:00 → 2:30 would clash with itself.

---

9. Cancelling an appointment

1. Customer: "Please cancel my appointment." The agent calls `propose_cancellation` .
2. The card shows `cancel_pending` with Cancel appointment / Keep it.
3. Cancel appointment → `apply_change` deletes the Google event (Google emails a cancellation), removes the booking and frees the slot. Keep it → `discard_change` ; nothing changes.

propose_cancellation · zovak/agents/receptionist.py

```
@function_tool
async def propose_cancellation(ctx: RunContextWrapper[AgentContext]) -> str:
    """Prepare cancelling THIS conversation's confirmed appointment. Cancels nothing.

    The customer must confirm the cancellation separately.
    """
    try:
        await ctx.context.scheduler.propose_cancel(
            ctx.context.tenant_id, ctx.context.conversation_id
        )
        return (
            "PENDING cancellation. NOT cancelled yet. Customer must explicitly "
            "confirm the cancellation through the confirmation action."
        )
    except SchedulingError as exc:
        return f"Cannot propose cancellation: {exc}"
```

DemoScheduler.propose_cancel · zovak/services/mock_scheduler.py

```
async def propose_cancel(self, tenant_id: str, conversation_id: str) -> ChangeRequest:
    """Prepare a cancellation ONLY. Nothing changes until apply_change()."""
    self._check_tenant(tenant_id)
    booking = await self._bookable_for_change(tenant_id, conversation_id)
    return await self._store_change(ChangeRequest(
        id=str(uuid4()), tenant_id=tenant_id, conversation_id=conversation_id,
        booking_id=booking.id, kind="cancel",
        expires_at=datetime.now(timezone.utc) + timedelta(minutes=5),
    ))
```

GoogleInvites.cancel_event · zovak/providers/calendar/google_invites.py

```
async def cancel_event(self, *, event_id: str) -> None:
    """Delete an event; Google emails the attendee a cancellation notice."""
    async with httpx.AsyncClient(timeout=15) if self._client is None else _shared(self._client) as
client:
        response = await client.delete(
            self._url + "/events/" + quote(event_id, safe=""),
            params={"sendUpdates": "all"}, headers=self._headers(),
        )
        if response.status_code in (404, 410):
            return  # already gone, e.g. a retry after a lost response
        response.raise_for_status()
```

VoiceSideband._commit_change · zovak/voice/sideband.py

```
async def _commit_change(self, change) -> str:
    """Apply a pending move/cancel. Caller holds the lock."""
    try:
        result = await self.scheduler.apply_change(_TENANT, self.conversation_id, change.id)
    except SchedulingError as exc:
        return f"I couldn't complete that change: {exc}. Your appointment is unchanged."
    self.wrap_up = True
    await note_for_agent(_TENANT, self.conversation_id, (
        "The customer confirmed the move; the booking is now at the new time."
        if result.status == "moved" else
        "The customer confirmed the cancellation; the booking is cancelled."
    ))
    if result.status == "cancelled":
        return (
            "Your appointment is cancelled. "
            + ("Google Calendar was asked to email the cancellation. "
               if result.provider == "google-calendar" else "")
            + _ANYTHING_ELSE.strip()
        )
    when = change.new_start.astimezone(self.scheduler.timezone)
    return (
        f"Done. Your appointment is moved to {when.strftime('%A, %B %d at %I:%M %p')}, "
        "Dhaka time. "
        + ("Google Calendar was asked to email the updated invitation."
           if result.provider == "google-calendar" else "")
        + _ANYTHING_ELSE
    )
```

10. Spoken confirmations

Speech arrives in pieces, so a bare "yes" might be the start of "yes, but change the time". The rules run in plain code, not the AI, and accept natural wording ("uh, yes, confirm it now please", "go ahead", "okay book it") while never treating a possibly-unfinished reply as a yes:

1. **An action is required.** "Yes" or "sure" alone is not enough; there must be "confirm", "book", "go ahead", "do it"…
2. **Hedge or change words block it:** but, wait, hold, change, actually, not, maybe, tomorrow, pm…
3. **Unfinished replies don't count:** one ending in "my", "the", "to"… ("yes, confirm my") is cut off.
4. **ZOVAK waits 1.5 s of silence** before acting. If the caller keeps talking, nothing happens; their next words are handled as a new turn.

_is_approval · zovak/voice/confirmation.py

```
def _is_approval(text: str, pattern: re.Pattern) -> bool:
    words = normalize_utterance(text.replace("thank you", "thanks")).split()
    if not words or any(w in _HEDGE for w in words):
        return False
    words = [w for w in words if w not in _FILLER]
    if not words or words[-1] in _DANGLING:
        return False
    utterance = " ".join(words)
    # An action is required: affirmation words alone ("yes", "sure") don't count.
    if re.fullmatch(rf"(?:{_AFFIRM}\s*)+", utterance):
        return False
    return bool(pattern.fullmatch(utterance))
```

classify_confirmation · zovak/voice/confirmation.py

```
def classify_confirmation(text: str) -> Decision:
    """Approve only an *entire*, unmistakably positive utterance."""
    utterance = normalize_utterance(text)
    if _DECLINE.fullmatch(utterance):
        return "decline"
    if _REVISION.search(utterance):
        return "revise"
    if is_approval(text, "book"):
        return "approve"
    return "unclear"
```

VoiceSideband._settled · zovak/voice/sideband.py

```
async def _settled(self) -> bool:
    """Wait for a moment of silence before acting on a spoken "yes".

    If the caller kept talking ("yes... but change the time"), it wasn't a
    complete yes: the new words arrive as their own turn instead.
    """
    heard = self._turn_heard_at
    if heard is None:  # not a live call (tests, direct calls)
        return True
    deadline = monotonic() + 4
    while monotonic() < deadline:
        if self._last_input_at > heard:
            return False
        if monotonic() - heard >= self.SETTLE_SECONDS:
            return True
        await asyncio.sleep(0.1)
    return self._last_input_at <= heard
```

classify_change_reply · zovak/voice/confirmation.py

```
def classify_change_reply(text: str, kind: str) -> Decision:
    """Classify a reply to 'shall I move/cancel your confirmed appointment?'."""
    utterance = normalize_utterance(text)
    if _KEEP.fullmatch(utterance):
        return "decline"
    if is_approval(text, kind):
        return "approve"
    if _REVISION.search(utterance):
        return "revise"
    return "unclear"
```

Every spoken turn goes through `_handle_turn`. It starts by looking up the trusted pending proposal for this call:

VoiceSideband._handle_turn · zovak/voice/sideband.py

```
async def _handle_turn(self, text: str, run_agent) -> str:
    """Coordinate voice confirmation without giving the LLM booking authority.

    Serialized by _agent_lock in _delegate. Uses the scheduler's trusted
    proposal records rather than an LLM-supplied proposal ID.
    """
    tenant = "demo-business"  # Local-only, hardcoded demo identity.
    if await read_choices(self.scheduler, tenant, self.conversation_id, text, emails=False):
        # "Change the time / doctor": the waiting proposal was set aside on purpose.
        self.confirmation.clear()
        self.email_form.clear()
    if is_goodbye(text):
… 182 more lines in the file
```

It then, in order: handles a pending proposal (email gate, approve, decline, revise); handles a pending move/cancel with `classify_change_reply`; otherwise runs the agent, applies the off-topic check, and announces any newly proposed booking or change.

11. Reminder emails

A background task checks every minute for confirmed bookings starting within the next hour and emails the customer once through Resend. It skips bookings made inside that hour, follows moved bookings (their reminder flag is reset) and never reminds cancelled ones. An idempotency key makes a retried send deliver only once.

ReminderWorker.run_once · zovak/workers/reminders.py

```
async def run_once(self, now: datetime | None = None) -> list[str]:
    """Send reminders that are due; returns the booking IDs reminded."""
    now = now or datetime.now(timezone.utc)
    sent = []
    for booking in list(self.scheduler._bookings.values()):
        if booking.status != "confirmed" or booking.reminder_sent or not booking.customer_email:
            continue
        if not now < booking.start <= now + self.lead:
            continue
        # Booked (or moved) inside the reminder window: they just got a
        # confirmation, so skip rather than send a near-duplicate.
        if booking.confirmed_at and booking.confirmed_at > booking.start - self.lead:
            booking.reminder_sent = True
            continue
        service = self.scheduler.services.get(booking.service_id)
        subject, page, text = reminder_email(
            booking, service.name if service else booking.service_id, self.scheduler.timezone,
        )
        try:
            await send_resend_email(
                self.api_key, self.sender, booking.customer_email, subject, page, text,
                idempotency_key=f"reminder-{booking.id}-{booking.start.isoformat()}",
                client=self.client,
            )
        except (httpx.HTTPError, RuntimeError):
            logger.warning("Reminder for booking %s failed; will retry", booking.id, exc_info=True)
            continue
        booking.reminder_sent = True
        sent.append(booking.id)
        logger.info("Sent appointment reminder for booking %s", booking.id)
    return sent
```

send_resend_email · zovak/providers/notifications/resend_email.py

```
async def send_resend_email(api_key: str, sender: str, recipient: str, subject: str,
                            html: str, text: str, *, idempotency_key: str,
                            client: httpx.AsyncClient | None = None) -> str:
    """Send one email. The idempotency key makes a retried send deliver only once."""
    request = {
        "url": "https://api.resend.com/emails",
        "headers": {"Authorization": f"Bearer {api_key}", "Idempotency-Key": idempotency_key},
        "json": {"from": sender, "to": [recipient], "subject": subject, "html": html, "text":
text},
    }
    if client is None:
        async with httpx.AsyncClient(timeout=15) as http:
            response = await http.post(**request)
    else:
        response = await client.post(**request)
    response.raise_for_status()
    message_id = response.json().get("id")
    if not message_id:
        raise RuntimeError("Resend did not return a message ID")
    return str(message_id)
```

12. Today's date and live booking status

The agent and GPT-Live are given today's date and time (Dhaka) on every chat turn and call, so Nofa resolves "tomorrow" or "26th October" itself and never asks for the year.

today_instructions · zovak/agents/receptionist.py

```
def today_instructions(tz: str = "Asia/Dhaka") -> str:
    """Today's date, so the agent resolves dates without asking for the year."""
    now = datetime.now(ZoneInfo(tz))
    return (
        f"Today is {now.strftime('%A, %B %d, %Y')} and the time is "
        f"{now.strftime('%I:%M %p')} ({tz}). Never ask the customer for the year. "
        "Resolve dates yourself: a date without a year means its next upcoming "
        "occurrence (this year, or next year if it has already passed). "
        "Words like today, tomorrow and next Friday are relative to today. "
    )
```

12.1 Why the agent needs the live status

Buttons and strict spoken confirmations change bookings *without* the agent. Its memory would still hold the old tool result ("PENDING… not confirmed"), so after a button confirmation it once refused to move a booking that was already confirmed. Two safeguards now prevent this:

1. Before every agent turn, `booking_status_for_agent` reads the scheduler and the status is put into the agent's instructions, marked as always current.
2. Whenever a booking is confirmed, moved, cancelled or kept outside the agent, `note_for_agent` adds a short system note to the agent's memory.

booking_status_for_agent · zovak/services/booking_state.py

```
async def booking_status_for_agent(scheduler, tenant_id: str, conversation_id: str,
                                   channel: str = "chat") -> str:
    """Plain-text status the agent gets on every turn, straight from the scheduler."""
    lines = []
    booking = await scheduler.latest_booking(tenant_id, conversation_id)
    proposal = await scheduler.latest_pending(tenant_id, conversation_id)
    change = await scheduler.latest_change(tenant_id, conversation_id)
    if booking is not None:
        service = scheduler.services.get(booking.service_id)
        name = service.name if service else booking.service_id
        lines.append(f"CONFIRMED booking: {name} with {scheduler.doctor_name(booking.doctor_id)} "
                     f"for {booking.customer_name}, {_when(scheduler, booking.start,
booking.end)}.")
    if proposal is not None:
        lines.append(f"PENDING proposal, not booked yet: {proposal.service_id} with "
                     f"{scheduler.doctor_name(proposal.doctor_id)} at "
                     f"{_when(scheduler, proposal.start, proposal.end)}; email "
                     f"{'entered' if proposal.customer_email else 'not entered yet'}.")
    if change is not None and change.kind == "move":
        lines.append(f"PENDING move, not applied yet: to "
                     f"{_when(scheduler, change.new_start, change.new_end)}.")
    elif change is not None:
        lines.append("PENDING cancellation, not applied yet.")
    if booking is None and scheduler.last_cancelled(tenant_id, conversation_id) is not None:
        lines.append("The booking made earlier in this conversation was cancelled.")
    # What the website shows right now, so Nofa never points at buttons that aren't there.
    chosen = getattr(scheduler, "chosen_doctor", lambda *_: None)(tenant_id, conversation_id)
    picked = getattr(scheduler, "picked_time", lambda *_: None)(tenant_id, conversation_id)
    carried = getattr(scheduler, "carried_details", lambda *_: None)(tenant_id, conversation_id)
    if proposal is None and carried:
        lines.append(f"The patient's name is already known: {carried[0]}. Don't ask for it again."
                     + (" Their email is saved too: call propose_booking WITHOUT customer_email "
                        "(the booking system adds it) and don't ask for it." if carried[1] else
""))
    moving = getattr(scheduler, "moving_booking", lambda *_: None)(tenant_id, conversation_id)
    if proposal is None and chosen and moving and channel in ("chat", "web"):
        lines.append(f"WEBSITE SCREEN: the patient is choosing a NEW time for their confirmed
booking "
                     f"with {scheduler.doctor_name(chosen)} (their one allowed change); the time
buttons "
                     "are showing. If they type a time instead of tapping, call propose_move, never
"
                     "propose_booking.")
    elif proposal is None and chosen and channel in ("chat", "web"):  # phone/WhatsApp have no
screen
        lines.append(f"WEBSITE SCREEN: {scheduler.doctor_name(chosen)} is chosen" + (
            f"; the patient picked {picked.astimezone(scheduler.timezone):%A, %B %d at %I:%M %p}
with the time buttons."
            if picked else "; their free times are showing as tap buttons."))
    return " ".join(lines) or "No appointment in this conversation yet."
```

note_for_agent · zovak/agents/memory.py

```
async def note_for_agent(tenant_id: str, conversation_id: str, text: str) -> None:
    """Record a booking-system event so the agent's next turn knows about it."""
    session = memory_session(tenant_id, conversation_id)
    try:
        await session.add_items([{"role": "system", "content": f"Booking system update: {text}"}])
    except Exception:
        logger.warning("Could not add a booking note to agent memory", exc_info=True)
    finally:
        await session.close()
```

13. Ending a call and guardrails

The same rules apply to website and phone calls. `CallGuard` is plain bookkeeping the sideband feeds with events.

When	What Nofa does
10 s of silence	Warns: "If I don't hear anything in the next 10 seconds, I'll end the call, and you won't be able to say anything after that."
10 s more of silence	Says goodbye and ends the call (website: 2 minutes while the email box is open)
An action finished	Asks "Is there anything else?"; "no, that's all" or "bye" ends the call
The caller says goodbye (any time)	"Thanks, bye", "I have to go", "you can hang up"… Nofa ends the call with a personal goodbye (the booked doctor, day and time, if any). Checked on every caller turn, even ones the voice model answers itself
Goodbye with a booking half-done	Asks once: "Before you go, your appointment isn't booked yet. Would you like to confirm it, or leave it for now?" A second goodbye ends the call; nothing is booked (a pending move/cancel is dropped too)
Off-topic requests	Refuses politely; the 3rd ends the call
Booking, moving, cancelling over and over	More than 4 prepared actions in one call, or the agent's `report_misuse`, ends the call at once
Too many calls from one caller	More than 5 calls an hour from the same caller ID are rejected (busy)
Only noise (cough, sniff, "uh", "hmm", [cough])	Doesn't reset the silence timer and isn't sent to the AI. Right after the warning, or on the 3rd noise-only turn, Nofa ends the call politely
Goodbye started	Nothing the caller says counts any more: the website mutes the mic, the backend ignores speech

CallGuard.tick · zovak/voice/call_guard.py

```
def tick(self, seconds: float, now: float, *, busy: bool, email_open: bool) -> Action:
    """Advance the silence count. busy = the backend is still working on a reply."""
    if self.ending or busy or now < self.speaking_until:
        return None
    self.idle += seconds
    limit = self.email_limit if email_open else self.silence_limit
    if self.idle >= limit:
        return "end"
    if not self.checked_in and self.idle >= limit - self.check_in_before:
        self.checked_in = True
        return "check_in"
    return None
```

CallGuard.count_action · zovak/voice/call_guard.py

```
def count_action(self) -> bool:
    """A new booking, move or cancellation was prepared. True when over the limit:
    someone booking, moving and cancelling over and over is wasting the line."""
    self.actions += 1
    return self.actions > self.action_limit
```

is_meaningful · zovak/voice/confirmation.py

```
def is_meaningful(text: str) -> bool:
    """True if the caller said real words, not just a cough, "uh" or "hmm".

    Works for any language: any word of 2+ letters (or a number) that isn't a
    known filler or noise counts.
    """
    words = re.findall(r"[^\W\d_]+|\d+", _NOISE_TAG.sub(" ", text).casefold())
    return any(w.isdigit() or (len(w) >= 2 and w not in _NOISE_WORDS) for w in words)
```

VoiceSideband._caller_said · zovak/voice/sideband.py

```
def _caller_said(self, words: str) -> None:
    """Only real words count as the caller being there: a cough, "uh" or "hmm"
    doesn't reset the silence timer (someone pretending to answer)."""
    if is_meaningful(words):
        self.guard.caller_active()
        self._last_input_at = monotonic()
```

VoiceSideband._noise_only_turn · zovak/voice/sideband.py

```
def _noise_only_turn(self) -> str:
    """Reply to a turn with no real words, without involving the agent."""
    if self.guard.noise_turn():
        logger.info("Ending call %s: noise only, no request", self.conversation_id)
        return self._end("noise")
    return _DIDNT_CATCH
```

VoiceSideband._end · zovak/voice/sideband.py

```
def _end(self, reason: str, message: str | None = None) -> str:
    """Say goodbye (returned as this turn's reply), then end the call."""
    goodbye = message or _GOODBYE[reason]
    self.guard.ending = True
    task = asyncio.create_task(self._finish_call(reason, goodbye))
    self._pending.add(task)
    task.add_done_callback(self._pending.discard)
    return goodbye
```

VoiceSideband._finish_call · zovak/voice/sideband.py

```
async def _finish_call(self, reason: str, goodbye: str) -> None:
    """Wait for the goodbye to be heard, then tell the website to hang up."""
    self.guard.ending = True
    await asyncio.sleep(speech_seconds(goodbye))
    self.call_ended = reason
    logger.info("Ending voice call %s: %s", self.conversation_id, reason)
    if self._hang_up is not None:  # phone: hang up the SIP call
        try:
            await self._hang_up()
        except Exception:
            logger.warning("Could not hang up phone call %s", self.session_id, exc_info=True)
        return
    # Give the website a couple of polls to see call_ended and hang up itself.
    await asyncio.sleep(self.HANGUP_GRACE)
    if self._socket is not None and not self.closed:
        try:  # best effort; the website also closes its side
            await self._socket.send(json.dumps({"type": "session.close"}))
        except Exception:
            logger.warning("Could not close GPT-Live session from the backend", exc_info=True)
```

is_goodbye · zovak/voice/confirmation.py

```
def is_goodbye(text: str) -> bool:
    """The caller clearly wants to end the call ("thanks, bye", "I have to go", "hang up")."""
    utterance = normalize_utterance(text)
    if not utterance or len(utterance.split()) > 12 or "?" in text:
        return False
    return bool(_GOODBYE.search(utterance)) and not _STILL_ASKING.search(utterance)
```

VoiceSideband._goodbye_turn · zovak/voice/sideband.py

```
async def _goodbye_turn(self) -> str:
    """The caller wants to finish: a warm goodbye, unless something is half-done."""
    pending = await self.scheduler.latest_pending(_TENANT, self.conversation_id)
    change = await self.scheduler.latest_change(_TENANT, self.conversation_id)
    if (pending or change) and not self._leave_warned:
        self._leave_warned = True
        return _LEAVE_PROPOSAL if pending else _LEAVE_CHANGE
    if pending:
        await self.scheduler.discard(_TENANT, self.conversation_id, pending.id)
        self.confirmation.clear()
        self.email_form.clear()
        return self._end("done", "No problem, I haven't booked anything. " + _THANKS)
    if change:
        await self.scheduler.discard_change(_TENANT, self.conversation_id, change.id)
        return self._end("done", "No problem, your appointment stays as it was. " + _THANKS)
    return self._end("done", await self._farewell())
```

VoiceSideband._farewell · zovak/voice/sideband.py

```
async def _farewell(self) -> str:
    """ "You're all set with Dr. Rahman on Friday at 11:30 AM. Thanks for calling…" """
    booking = await self.scheduler.latest_booking(_TENANT, self.conversation_id)
    if booking is None:
        return _THANKS
    start = booking.start.astimezone(self.scheduler.timezone)
    days = (start.date() - datetime.now(self.scheduler.timezone).date()).days
    day = {0: "today", 1: "tomorrow"}.get(days, f"on {start:%A, %B} {start.day}")
    doctor = self.scheduler.doctors.get(booking.doctor_id or "")
    with_doctor = f" with {doctor.name}" if doctor else ""
    time_of_day = start.strftime("%I:%M %p").lstrip("0")
    return f"You're all set{with_doctor} {day} at {time_of_day}. {_THANKS}"
```

VoiceSideband._check_goodbye · zovak/voice/sideband.py

```
async def _check_goodbye(self) -> None:
    """Fallback: the voice model answered "bye" itself instead of asking ZOVAK.

    Runs after each fragment; only the one after the caller's last words acts.
    Words already handed to ZOVAK by a delegation are skipped.
    """
    await asyncio.sleep(self.GOODBYE_WAIT)
    if (monotonic() - self._last_heard_at < self.GOODBYE_WAIT or self.guard.ending
            or self.closed or self._agent_lock.locked()):
        return
    fragments = self.transcripts.input_fragments
    since = max(self.transcripts.last_delegation_ms, self._goodbye_checked_ms)
    text = "".join(words for end, words in fragments if end > since).strip()
    if fragments:
        self._goodbye_checked_ms = fragments[-1][0]
    if not is_goodbye(text):
        return
    async with self._agent_lock:
        if self.guard.ending:
            return
        self.transcripts.last_delegation_ms = max(self.transcripts.last_delegation_ms,
                                                  self._goodbye_checked_ms)
        reply = await self._goodbye_turn()
    logger.info("Caller said goodbye (not delegated): %r", text[:80])
    await self._speak(reply)
```

report_misuse · zovak/agents/receptionist.py

```
@function_tool
async def report_misuse(ctx: RunContextWrapper[AgentContext]) -> str:
    """Call this when the customer is clearly wasting time: repeatedly booking, moving
    and cancelling with no real intent, testing or tricking you, or being abusive.
    Do NOT call it for a genuine customer who simply changed their mind once or twice."""
    if ctx.context.on_misuse is not None:
        ctx.context.on_misuse()
    return "The backend will end the call politely."
```

The website sees `call_ended` (or `ending`) in the polled state, mutes the mic and hangs up:

endedRemotely · serena-health/src/lib/agent/client.ts

```
const endedRemotely = (reason: CallEndReason) => {
  if (stopped) return;
  stopped = true;
  if (channel?.readyState === "open") channel.send(JSON.stringify({ type: "session.close" }));
  cleanup();
  handlers.onCallEnded?.(reason);
  handlers.onStateChange("idle");
};
```

14. Phone calls (SIP, Telnyx)

A phone call reaches OpenAI GPT-Live over SIP. OpenAI asks ZOVAK to accept it; after that ZOVAK runs the call with the same sideband as the website. Nothing in ZOVAK depends on who carries the call, so the Telnyx test number now and a real Bangladeshi number later use the same code.

Under the hood: how a phone call connects

Connection	Between	Carries
Phone network	Caller's phone ↔ Telnyx	The normal phone call
SIP over TLS (signaling)	Telnyx → OpenAI (`sip:proj_…@sip.api.openai.com:5061`)	"Here's a call for project proj_…" — the Translated Number puts the project ID there
SRTP (encrypted audio)	Telnyx ↔ OpenAI	The voices. Telnyx's public servers make this work; a phone app behind a router (Linphone) couldn't
Signed webhook (HTTPS)	OpenAI → ZOVAK (through ngrok)	`live.transport.incoming` : session ID, SIP headers (caller ID), audio security
Accept (HTTPS)	ZOVAK → OpenAI	`POST /v1/live/sessions/{id}/accept` with Nofa's phone instructions
Sideband WebSocket	ZOVAK ↔ OpenAI	Same as the website: transcripts, delegation, replies, hang-up

1. The caller dials the Telnyx number. Telnyx sends a SIP INVITE over TLS to OpenAI, with the user part translated to your `proj_…` ID, and negotiates SRTP audio.
2. OpenAI holds the call and sends ZOVAK a **signed** `live.transport.incoming` webhook. ZOVAK verifies the signature with `OPENAI_WEBHOOK_SECRET` (a forged request is refused with 400), ignores repeats of the same event, and rate-limits the caller.
3. ZOVAK accepts the session (the ID is passed back unchanged) with Nofa's phone instructions, then attaches the sideband (`channel=phone`). From here the call works exactly like a website call, except confirming sends the SMS form, and ending the call is a real hang-up (`live.sessions.hangup`).
4. If ZOVAK doesn't accept (server down, wrong webhook URL), OpenAI can't connect the call and the caller hears nothing.

Telnyx number (now) / Bangladeshi number (later)

Carrier sends the call to sip:proj_…@sip.api.openai.com over TLS with SRTP audio

↓

OpenAI GPT-Live

Sends a signed live.transport.incoming webhook to ZOVAK (through ngrok)

↓

ZOVAK /api/v1/phone/openai-webhook

Verifies the signature, ignores duplicates, rate-limits the caller, accepts with Nofa's phone instructions

↓

VoiceSideband (channel=phone)

Same booking rules, silence and guardrails; confirm sends the SMS form; ends with a real hang-up

openai_webhook · zovak/api/routes/phone.py

```
@router.post("/openai-webhook")
async def openai_webhook(request: Request) -> dict:
    settings = request.app.state.settings
    if not settings.openai_webhook_secret or not settings.openai_api_key:
        raise HTTPException(503, detail="Set OPENAI_WEBHOOK_SECRET and OPENAI_API_KEY in .env")
    client = openai_client(settings.openai_api_key)
    try:
        event = client.webhooks.unwrap(
            await request.body(), request.headers, secret=settings.openai_webhook_secret,
        )
    except (InvalidWebhookSignatureError, ValueError) as exc:
        raise HTTPException(400, detail="Invalid webhook signature") from exc

    if event.id in _seen_events:
        return {"ok": True, "duplicate": True}
    _seen_events[event.id] = None
    while len(_seen_events) > 1000:
        _seen_events.popitem(last=False)
    if event.type != "live.transport.incoming":
        return {"ok": True, "ignored": event.type}

    session_id = event.data.session_id  # forward unchanged
    caller = caller_id(event.data.sip_headers)
    if _too_many_calls(caller, settings.max_calls_per_caller_per_hour):
        logger.warning("Rejecting call from %s: too many calls this hour", caller)
        await client.live.sessions.reject(session_id, status_code=486)  # busy here
        return {"ok": True, "rejected": "rate_limited"}

    try:
        await client.live.sessions.accept(session_id, session={
            "type": "live",
            "model": "gpt-live-1",
            "instructions": PHONE_PROMPT + " " + today_instructions(),
            "delegation": {"type": "client"},
        })
    except APIError:
        logger.exception("Could not accept phone call %s", session_id)
        return {"ok": True, "accepted": False}

    if not hasattr(request.app.state, "voice_sidebands"):
        request.app.state.voice_sidebands = {}
    active: dict = request.app.state.voice_sidebands

    async def hang_up() -> None:
        await openai_client(settings.openai_api_key).live.sessions.hangup(session_id)

    handler = VoiceSideband(
        session_id=session_id,
        conversation_id=f"phone-{uuid4().hex}",
        api_key=settings.openai_api_key,
        scheduler=request.app.state.scheduler,
        agent_model=settings.openai_agent_model,
        on_close=lambda sid: active.pop(sid, None),
        channel="phone",
        caller=caller,
        booking_forms=request.app.state.booking_forms,
        sms=request.app.state.sms,
        public_base_url=settings.public_base_url or "http://127.0.0.1:8000",
        hang_up=hang_up,
    )
    active[session_id] = handler
    handler.start()
```

```
    logger.info("Accepted phone call %s from %s (call audio: %s)", session_id, caller,
                event.data.sip_media_security or "unknown")
    if event.data.sip_media_security == "rtp":
        logger.warning("Call audio is NOT encrypted (rtp); GPT-Live needs SRTP, expect no sound")
    return {"ok": True, "accepted": True}
```

caller_id · zovak/api/routes/phone.py

```
def caller_id(sip_headers) -> str:
    """Caller from the SIP From header, e.g. '+8801712345678' or 'alice@sip.linphone.org'.

    Untrusted metadata: fine for addressing a text, NOT proof of identity.
    """
    value = next((h.value for h in sip_headers if h.name.lower() in ("from", "f")), "")
    match = re.search(r"<?sips?:([^@>;]+)(@[^>;]+)?", value)
    if not match:
        return "unknown"
    user, host = match.group(1), (match.group(2) or "")
    return user if re.fullmatch(r"\+?\d{6,15}", user) else f"{user}{host}"
```

VoiceSideband._phone_turn · zovak/voice/sideband.py

```
async def _phone_turn(self, text: str, current) -> str | None:
    """A reply while a proposal waits on a phone call. None = let the agent handle it."""
    decision = classify_confirmation(text)
    if decision == "approve":
        if not await self._settled():
            return _NOT_SETTLED
        return await self._send_form(current)
    if decision == "decline":
        await self.scheduler.discard(_TENANT, self.conversation_id, current.id)
        self.confirmation.clear()
        self.wrap_up = True
        return "Okay, I've dropped that appointment. Nothing was booked." + _ANYTHING_ELSE
    if decision == "revise":
        await self.scheduler.discard(_TENANT, self.conversation_id, current.id)
        self.confirmation.clear()
        return None
    return self._phone_question(current)
```

14.1 Diagnosing a call with no sound

The uvicorn terminal shows ZOVAK's own timestamped lines. A healthy call looks like:

```
Accepted phone call live_… from alice@sip.linphone.org (call audio: srtp)
GPT-Live sideband connected for phone call phone-…
Nofa is speaking (1.2 s after connect)       ← audio is being generated
Caller audio is arriving (4.8 s after connect) ← your voice reaches OpenAI
Caller said: 'I'd like to book a consultation tomorrow'
Nofa replies: '…'
```

What the log shows	Meaning
`call audio: rtp`	Linphone didn't use encryption; GPT-Live needs SRTP
No "Nofa is speaking"	Nofa never produced a reply/greeting
"Nofa is speaking" but you hear nothing	Audio from OpenAI isn't reaching the phone (network/NAT)
No "Caller audio is arriving"	Your voice isn't reaching OpenAI; after 45 s ZOVAK ends the call as `no_audio`

14.2 Only the phone route and forms are public

ngrok exposes port 8000, so ZOVAK refuses every request arriving through the tunnel except the webhook and the form pages. The website talks to ZOVAK on 127.0.0.1 and is unaffected.

_guard_public_host · zovak/main.py

```
def _guard_public_host(app: FastAPI, public_base_url: str, site_api_key: str = "") -> None:
    """Requests arriving on the public host can only reach the phone webhook and forms.

    The website and its Next.js routes call ZOVAK on 127.0.0.1, so they are unaffected.
    A deployed website (e.g. on Vercel) reaches the other routes through the tunnel only by
    sending the shared site key in X-Zovak-Site-Key; nobody else can.
    """
    public_host = urlparse(public_base_url).hostname if public_base_url else None
    if not public_host:
        return

    @app.middleware("http")
    async def public_only(request: Request, call_next):
        host = (request.headers.get("host") or "").split(":")[0].lower()
        # Tunnels (ngrok) add X-Forwarded-For; local calls from the website don't.
        public = host == public_host.lower() or "x-forwarded-for" in request.headers
        sent_key = request.headers.get("x-zovak-site-key", "")
        from_site = bool(site_api_key) and compare_digest(sent_key.encode(), site_api_key.encode())
        if public and not from_site and not request.url.path.startswith(PUBLIC_PATHS):
            return PlainTextResponse("Not found", status_code=404)
        return await call_next(request)
```

14.3 Working setup: Telnyx number → OpenAI (tested 2026-10-27)

Calling OpenAI straight from a phone app (Linphone) connected but no audio ever flowed: phones behind home routers or mobile networks can't receive the call audio. A SIP provider carries the audio from public servers, which is what OpenAI expects. This setup worked end to end:

1. Telnyx: upgrade the account (a trial can only buy a number in your home country), then buy a US **local** number. Toll-free numbers can't be called from abroad.
2. Voice → SIP Trunking → new **FQDN** connection "OpenAI GPT-Live": FQDN `sip.api.openai.com`, port `5061`, DNS record `A`, set as Primary FQDN.
3. Inbound tab: SIP transport protocol **TLS**; codecs Opus, G711U, G711A. Configuration tab: Encrypted media **SRTP Mandatory** (it only saves once TLS is set).
4. Phone Numbers → the number → Voice tab: SIP Connection = OpenAI GPT-Live; Advanced → **Translated Number** = your `proj_…` ID; Save Changes.

5. OpenAI → Settings → Webhooks: `https://YOUR-NGROK/api/v1/phone/openai-webhook`, event `live.transport.incoming` (not `realtime.call.incoming`, which is the older Realtime API); signing secret into `OPENAI_WEBHOOK_SECRET`.
6. Call the number. Say "confirm", then open the form from `/api/v1/dev/sms-outbox`.

14.4 Linphone direct (didn't work: no audio)

Kept for reference. The call was accepted and ZOVAK attached, but no audio packets ever flowed (red signal in Linphone, no "Nofa is speaking", nothing billed), and the call closed after ~30 s.

1. OpenAI dashboard: check GPT-Live SIP is enabled for your project and copy the project ID (`proj_…`).
2. Run `ngrok http 8000` and copy the https address.
3. OpenAI dashboard → Webhooks: add `https://YOUR-NGROK/api/v1/phone/openai-webhook`, subscribe to `live.transport.incoming`, copy the signing secret.
4. `.env`: `OPENAI_WEBHOOK_SECRET=whsec_…` and `PUBLIC_BASE_URL=https://YOUR-NGROK`; restart uvicorn and reconnect Google.
5. Linphone: create or use a SIP account, set transport to TLS and media encryption to SRTP (mandatory).
6. Dial `sip:proj_YOUR_ID@sip.api.openai.com:5061;transport=tls`, book, say "confirm".
7. Open `http://127.0.0.1:8000/api/v1/dev/sms-outbox` on your laptop and tap the form link (it also works on your phone through ngrok).

---

15. The SMS booking form

1. On the call, Nofa reads the details back. Saying "confirm" (a safe yes) does NOT book: it creates a one-time form and texts the link.
2. Nofa says: "I've sent you a text… If you want to change the slot or cancel the booking, you can still do it from the form. I'll end the call now." and hangs up.
3. The form shows a 2:00 countdown; the slot is held for that time (other callers can't take it).
4. The customer can edit the name, must add an email, may pick another free time, then taps Confirm appointment, or Cancel booking.
5. Confirm books it (Google invite to the email). The link then only shows the result: it works once.

BookingForms.submit · zovak/services/booking_forms.py

```
async def submit(self, token: str, *, action: str, name: str = "", email: str = "",
                 start_iso: str = "") -> BookingForm:
    form = self.get(token)
    if form is None:
        raise SchedulingError("This form link is not valid")
    if form.status != "open":
        raise SchedulingError({
            "confirmed": "This form was already used to confirm the appointment",
            "cancelled": "This form was already used to cancel",
            "expired": "This form expired. Please call again to book",
        }[form.status])
    scheduler, tenant, conv = self.scheduler, form.tenant_id, form.conversation_id

    if action == "cancel":
        await scheduler.discard(tenant, conv, form.proposal_id)
        form.status = "cancelled"
        await note_for_agent(tenant, conv, "The customer cancelled using the SMS form.")
        return form
    if action != "confirm":
        raise SchedulingError("Unknown action")

    name = " ".join(name.split())
    if not 1 <= len(name) <= 80:
        raise ValueError("Please enter your name")
    email = validate_typed_email(email.strip())
    proposal = self.proposal(form)
    if proposal is None or proposal.status != "pending":
        raise SchedulingError("This appointment is no longer held. Please call again to book")
    start = datetime.fromisoformat(start_iso) if start_iso else proposal.start
    if start != proposal.start:
        # A different free time: replace the draft, then confirm that one.
        proposal = await scheduler.propose(tenant, conv, form.service_id, start, name,
                                           customer_email=email, doctor_id=proposal.doctor_id,
                                           customer_phone=proposal.customer_phone)
        form.proposal_id = proposal.id
    else:
        proposal.customer_name = name
        await scheduler.set_proposal_email(tenant, conv, proposal.id, email)
    result = await scheduler.confirm(tenant, conv, proposal.id)
    form.status = "confirmed"
    form.booking_id = result.booking_id
    await note_for_agent(tenant, conv, "The customer confirmed the appointment using the SMS
form.")
    return form
```

VoiceSideband._send_form · zovak/voice/sideband.py

```
async def _send_form(self, proposal) -> str:
    """Spoken "confirm" on a call sends the form; submitting it is what books."""
    form = await self.booking_forms.create(_TENANT, proposal, self.caller or "unknown")
    link = f"{self.public_base_url.rstrip('/')}/form/{form.token}"
    service = self.scheduler.services.get(proposal.service_id)
    when = proposal.start.astimezone(self.scheduler.timezone)
    await self.sms.send(self.caller or "unknown", (
        f"Nofa: confirm your {service.name if service else proposal.service_id} on "
        f"{when.strftime('%a %b %d, %I:%M %p')} within 2 minutes: {link} "
        "You can also pick another time or cancel there."
    ))
    self.confirmation.clear()
    await note_for_agent(_TENANT, self.conversation_id,
                         "The booking form was texted to the customer; the call is ending.")
    return self._end("form_sent", _FORM_SENT)
```

DemoScheduler._held_by_other · zovak/services/mock_scheduler.py

```
def _held_by_other(self, tenant_id: str, start: datetime, end: datetime,
                   conversation_id: str | None, doctor_id: str | None = None) -> bool:
    """A pending proposal in ANOTHER conversation holds its doctor's slot until it expires."""
    if conversation_id is None:
        return False
    now = datetime.now(timezone.utc)
    return any(
        p.tenant_id == tenant_id and p.conversation_id != conversation_id
        and p.status == "pending" and p.expires_at >= now
        and self._same_doctor(p.doctor_id, doctor_id)
        and start < p.end and end > p.start
        for p in self._proposals.values()
    )
```

15.1 Where a text actually goes: the SMS router

Nofa and the form only ever call `send(to, text)`. The router decides the rest, so adding or swapping a provider never touches the call or form code:

Destination	What happens	Setting
Test inbox	Always: `/api/v1/dev/sms-outbox` on your laptop	—
ntfy push (testing)	Free, instant notification on your phone; tapping it opens the form	`TEST_SMS_NTFY_TOPIC`
Telnyx (real SMS)	Any phone number except the skipped prefixes. Not reliable without a registered sender: a Sweden test was accepted, then rejected by the carrier (40008)	`TELNYX_API_KEY`, `TELNYX_FROM_NUMBER`, `TELNYX_MESSAGING_PROFILE_ID`
Bangladesh (+880)	Skipped by Telnyx (needs a registered sender: errors 40305/40306); test inbox + ntfy only, until a local gateway is added	`TELNYX_SMS_SKIP_PREFIXES=+880`
India	Not set up: needs DLT registration (Indian entity, approved templates)	—

A provider that fails or rejects a text is logged ("SMS to … via telnyx failed: …"); the text is still in the test inbox. Callers without a phone number (e.g. a SIP app address) only get the test inbox. To add a provider later: write a class with `async send(to, body)` and register it, e.g. `sms.add_provider("alpha-sms", AlphaSms(...), prefixes=("+880",))`.

SmsRouter.send · zovak/providers/notifications/sms_router.py

```
async def send(self, to: str, body: str) -> None:
    await self.outbox.send(to, body)
    if self.test_push is not None:
        try:
            await self.test_push.send(to, body)
        except Exception:
            logger.warning("Test push (ntfy) failed", exc_info=True)
    if not _E164.fullmatch(to):
        logger.info("No real SMS: %r isn't a phone number (test inbox only)", to)
        return
    match = self.provider_for(to)
    if match is None:
        logger.info("No SMS provider for %s yet (test inbox only)", to)
        return
    name, sender = match
    try:
        message_id = await sender.send(to, body)
        logger.info("SMS to %s accepted by %s (%s)", to, name, message_id)
    except Exception as exc:
        logger.warning("SMS to %s via %s failed: %s", to, name, exc)
```

NtfyPush.send · zovak/providers/notifications/ntfy_push.py

```
async def send(self, to: str, body: str) -> str:
    headers = {"Title": f"Nofa SMS to {to}", "Tags": "calendar"}
    link = _LINK.search(body)
    if link:
        headers["Click"] = link.group(0)  # tapping the notification opens the form
    if self.client is None:
        async with httpx.AsyncClient(timeout=10) as http:
            response = await http.post(self.url, content=body.encode(), headers=headers)
    else:
        response = await self.client.post(self.url, content=body.encode(), headers=headers)
    response.raise_for_status()
    return str(response.json().get("id", ""))
```

TelnyxSms.send · zovak/providers/notifications/telnyx_sms.py

```
async def send(self, to: str, body: str) -> str:
    payload = {"from": self.from_number, "to": to, "text": body}
    if self.messaging_profile_id:
        payload["messaging_profile_id"] = self.messaging_profile_id
    request = {
        "url": "https://api.telnyx.com/v2/messages",
        "headers": {"Authorization": f"Bearer {self.api_key}"},
        "json": payload,
    }
    if self.client is None:
        async with httpx.AsyncClient(timeout=15) as http:
            response = await http.post(**request)
    else:
        response = await self.client.post(**request)
    data = response.json() if response.content else {}
    if response.status_code >= 400 or data.get("errors"):
        error = (data.get("errors") or [{}])[0]
        raise RuntimeError(f"Telnyx {error.get('code', response.status_code)}: "
                           f"{error.get('detail') or error.get('title') or response.text[:200]}")
    return str(data["data"]["id"])
```

lifespan · zovak/main.py

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    from zovak.providers.notifications.ntfy_push import NtfyPush
    from zovak.providers.notifications.sms_router import SmsRouter
    from zovak.providers.notifications.telnyx_sms import TelnyxSms
    from zovak.services.booking_forms import BookingForms
    from zovak.services.google_scheduler import GoogleBackedScheduler
    from zovak.workers.reminders import ReminderWorker
    # Load the AI library now (~2.5 s), not during the first patient's first message.
    import agents  # noqa: F401
    from agents.extensions.memory import AsyncSQLiteSession  # noqa: F401
    import zovak.agents.receptionist  # noqa: F401
    app.state.scheduler = GoogleBackedScheduler(app)
    settings = app.state.settings = get_settings()
    if settings.demo_seed:
        from zovak.services.clinic import seed_demo_bookings
        logger.info("Loaded %d demo bookings for the demo doctors",
seed_demo_bookings(app.state.scheduler))
    app.state.booking_forms = BookingForms(app.state.scheduler,
minutes=settings.booking_form_minutes)
    app.state.sms = sms = SmsRouter()
    if settings.test_sms_ntfy_topic:
        sms.test_push = NtfyPush(settings.test_sms_ntfy_topic, settings.ntfy_server)
    if settings.telnyx_api_key and settings.telnyx_from_number:
        skip = tuple(p.strip() for p in settings.telnyx_sms_skip_prefixes.split(",") if p.strip())
        sms.add_provider("telnyx", TelnyxSms(
            settings.telnyx_api_key, settings.telnyx_from_number,
            settings.telnyx_messaging_profile_id,
        ), skip=skip)
    if settings.whatsapp_access_token and settings.whatsapp_phone_number_id:
        from zovak.providers.notifications.whatsapp_meta import WhatsAppCloud
        from zovak.services.whatsapp_bot import WhatsAppBot
… 36 more lines in the file
```

16. WhatsApp chat and calls

16.1 Chatbot

Each WhatsApp user is one conversation (`wa-<number>`), answered by the same receptionist agent as the website chat. Tap buttons replace the website card.

1. Meta sends each message to `/api/v1/whatsapp/webhook`, signed with `X-Hub-Signature-256` (HMAC-SHA256 of the body with the app secret). ZOVAK checks it, replies 200 at once and processes in the background; repeated deliveries are ignored.
2. Text goes to the agent (`channel=whatsapp`). If a booking is waiting for an email and the message contains one, ZOVAK attaches it directly.
3. When a booking, move or cancellation is pending, ZOVAK sends tap buttons: Confirm booking / Change email, Confirm move / Keep current time, Cancel appointment / Keep it. Button IDs carry the pending item's ID and are checked against it.
4. While the agent works, the message is marked read and WhatsApp shows "typing…" once (WhatsApp hides it after 25 s or when Nofa replies; no repeated refreshes, to keep API traffic human-paced).
5. Guardrails: 30 messages an hour per user, more than 4 prepared changes a day stops changes, images and voice notes get "please type".

WhatsAppBot._respond · zovak/services/whatsapp_bot.py

```
async def _respond(self, sender: str, message: dict) -> None:
    if self._rate_limited(sender):
        if sender not in self._warned:
            self._warned.add(sender)
            await self.cloud.send_text(sender, _TOO_MANY)
        return
    conversation = f"wa-{sender}"
    kind = message.get("type")
    if kind == "interactive" and message.get("interactive", {}).get("type") == "button_reply":
        await self._button(sender, conversation, message["interactive"]["button_reply"]["id"])
        return
    if kind != "text":
        await self.cloud.send_text(sender, _ONLY_TEXT)
        return
    text = message.get("text", {}).get("body", "").strip()
    logger.info("WhatsApp %s said: %r", sender, text[:160])
    if await self._take_email(sender, conversation, text):
        return
    reply = await self.run_agent(self.scheduler, TENANT, conversation, text, channel="whatsapp",
                                 customer_phone=f"+{sender}")
    await self.cloud.send_text(sender, reply)
    await self._show_card(sender, conversation)
```

WhatsAppBot._button · zovak/services/whatsapp_bot.py

```
async def _button(self, sender: str, conversation: str, button_id: str) -> None:
    action, _, item_id = button_id.partition(":")
    scheduler = self.scheduler
    pending = await scheduler.latest_pending(TENANT, conversation)
    change = await scheduler.latest_change(TENANT, conversation)
    try:
        if action in ("book", "email"):
            if pending is None or pending.id != item_id:
                await self.cloud.send_text(sender, _STALE)
                return
            if action == "email":
                await scheduler.clear_proposal_email(TENANT, conversation, pending.id)
                await self._show_card(sender, conversation)
                return
            result = await scheduler.confirm(TENANT, conversation, pending.id)
            self._last_card.pop(conversation, None)
            await note_for_agent(TENANT, conversation, "The customer tapped Confirm booking. "
                                                       "The appointment is now CONFIRMED.")
            invite = (" A calendar invitation is on its way to your email."
                      if result.provider == "google-calendar" else "")
            await self.cloud.send_text(sender, f"✅ You're booked:
{_when(pending.start.isoformat(), scheduler.timezone)}."
                                               + invite + _ANYTHING_ELSE)
            return
        if change is None or change.id != item_id:
            await self.cloud.send_text(sender, _STALE)
            return
        self._last_card.pop(conversation, None)
        if action == "keep":
            await scheduler.discard_change(TENANT, conversation, change.id)
            await note_for_agent(TENANT, conversation, "The customer chose to keep the appointment
unchanged.")
… 15 more lines in the file
```

receive · zovak/api/routes/whatsapp.py

```
@router.post("/webhook")
async def receive(request: Request) -> dict:
    settings = request.app.state.settings
    bot = getattr(request.app.state, "whatsapp", None)
    if bot is None or not settings.whatsapp_app_secret:
        raise HTTPException(503, detail="WhatsApp is not configured in .env")
    body = await request.body()
    if not signature_ok(body, request.headers.get("x-hub-signature-256", ""),
settings.whatsapp_app_secret):
        raise HTTPException(401, detail="Invalid signature")
    payload = json.loads(body)
    handlers = [bot.handle_payload(payload)]
    calls = getattr(request.app.state, "whatsapp_calls", None)
    if calls is not None:
        handlers.append(calls.handle_payload(payload))
    for work in handlers:
        task = asyncio.create_task(work)
        _tasks.add(task)
        task.add_done_callback(_tasks.discard)
    return {"ok": True}
```

16.2 Calls through the audio bridge

WhatsApp user-initiated calls are free for the caller. Three routes didn't work: Meta SIP straight to OpenAI (Meta puts the business number in the Request-URI, expects a digest challenge and the first SRTP packet from our side); Telnyx WhatsApp calling (needs a 2,000-recipient messaging limit); and handing Meta's WebRTC offer straight to GPT-Live — the logged SDPs show **both sides are ICE-lite** (`a=ice-lite`), so neither starts the connection checks and no audio ever flows. A browser works with GPT-Live because it's a full WebRTC client; the bridge plays that role on both sides.

WhatsApp caller → Meta
Meta's calls webhook: connect event with its SDP offer (ICE-lite, setup:actpass)

↓

Bridge leg 1 (aiortc)
Answers Meta's offer as a full ICE agent and starts the checks; sends silence until Nofa speaks

↓

Bridge leg 2 (aiortc)
Makes its own offer to GPT-Live (audio + oai-events data channel), like the website's browser

↓

Relay
Copies decoded audio frames both ways, resampled to mono 48 kHz, paced at real time, backlog capped

↓

Sideband
Attaches to the GPT-Live session: same phone rules, form link sent on WhatsApp

1. Meta sends `connect` with the caller's SDP offer.
2. ZOVAK makes the bridge's offer to GPT-Live (`live.create`, WebRTC) and applies GPT-Live's answer.
3. ZOVAK answers Meta's offer with the bridge's other leg and sends that answer with `pre_accept` then `accept`.
4. Audio flows caller ↔ Meta ↔ bridge ↔ GPT-Live. Every 5 s the log shows frames relayed each way. Hanging up closes both legs.
5. The call uses the same conversation as the caller's WhatsApp chat (`wa-<number>`), so bookings and Nofa's memory are shared between chat and calls.

WhatsAppCalls._connect · zovak/services/whatsapp_calls.py

```
async def _connect(self, call: dict) -> None:
    call_id, caller = call.get("id", ""), "+" + str(call.get("from", "")).lstrip("+")
    offer = (call.get("session") or {}).get("sdp")
    if not call_id or not offer or call_id in self.by_call:
        return
    settings = self.state.settings
    if _too_many_calls(caller, settings.max_calls_per_caller_per_hour):
        await self.cloud.call_action(call_id, "reject")
        return
    client = self._client()
    bridge = self._new_bridge(call_id[-10:])
    try:
        live = await client.live.create(
            session={"model": "gpt-live-1",
                     "instructions": WHATSAPP_PROMPT + " " + today_instructions(),
                     "delegation": {"type": "client"}},
            transport={"type": "webrtc", "sdp": await bridge.openai_offer()},
        )
        await bridge.openai_answer(live.transport.sdp)
        answer = await bridge.meta_answer(offer)
    except Exception:
        logger.exception("Could not set up the WhatsApp audio bridge")
        await bridge.close()
        await self.cloud.call_action(call_id, "reject")
        return
    logger.info("WhatsApp offer (Meta):  %s", sdp_summary(offer))
    logger.info("Bridge answer to Meta:  %s", sdp_summary(answer))
    await self.cloud.call_action(call_id, "pre_accept", answer)
    await self.cloud.call_action(call_id, "accept", answer)
    self.bridges[call_id] = bridge
    bridge.start_stats()
    logger.info("Accepted WhatsApp call %s from %s (GPT-Live session %s)",
                call_id, caller, live.session.id)

    async def hang_up() -> None:
        try:
            await self.cloud.call_action(call_id, "terminate")
        finally:
            handler = self.by_call.get(call_id)
            if handler is not None and handler._socket is not None:
                await handler._socket.send('{"type": "session.close"}')
            await self._close_bridge(call_id)

    # Same conversation as this person's WhatsApp chat (WhatsAppBot uses wa-<number>):
    # chat and calls share bookings and Nofa's memory. Meta verifies WhatsApp numbers,
    # unlike phone caller ID, so this link is safe.
    handler = VoiceSideband(
        session_id=live.session.id, conversation_id=f"wa-{caller.lstrip('+')}",
        api_key=settings.openai_api_key, scheduler=self.state.scheduler,
        agent_model=settings.openai_agent_model,
        on_close=lambda sid: self.by_call.pop(call_id, None),
        channel="phone", caller=caller, booking_forms=self.state.booking_forms,
        sms=WhatsAppOrSms(self.cloud, self.state.sms),
        public_base_url=settings.public_base_url or "http://127.0.0.1:8000",
        hang_up=hang_up,
    )
    self.by_call[call_id] = handler
    handler.start()
```

RelayTrack · zovak/voice/webrtc_bridge.py

```
class RelayTrack(MediaStreamTrack):
    """An outgoing audio track fed from the other leg, paced at real time.

    Sends silence until audio arrives (Meta expects our side to send first),
    and drops backlog so latency can't build up.
    """

    kind = "audio"

    def __init__(self):
        super().__init__()
        self.queue: asyncio.Queue[av.AudioFrame] = asyncio.Queue(maxsize=10)
        # One fixed format for everything we send: the Opus encoder can't switch
        # between our mono silence and the other leg's (often stereo) audio mid-call.
        self._resampler = av.AudioResampler(format="s16", layout="mono", rate=SAMPLE_RATE)
        self._start: float | None = None
        self._pts = 0
        self.forwarded = 0

    def push(self, frame: av.AudioFrame) -> None:
        frame.pts = None  # let the resampler restart timing; recv() sets our own pts
        for converted in self._resampler.resample(frame):
            if self.queue.full():
                self.queue.get_nowait()  # drop the oldest frame: stay real-time
            self.queue.put_nowait(converted)

    async def recv(self) -> av.AudioFrame:
        if self._start is None:
            self._start = time.monotonic()
        wait = self._start + self._pts / SAMPLE_RATE - time.monotonic()
        if wait > 0:
            await asyncio.sleep(wait)
        try:
            frame = self.queue.get_nowait()
            self.forwarded += 1
        except asyncio.QueueEmpty:
            frame = silence()
        frame.pts, frame.time_base = self._pts, _TIME_BASE
        self._pts += frame.samples
        return frame
```

AudioBridge.meta_answer · zovak/voice/webrtc_bridge.py

```
async def meta_answer(self, offer_sdp: str) -> str:
    """Leg 1: answer Meta's offer. As a full ICE agent we start the checks Meta waits for."""
    await self.meta.setRemoteDescription(RTCSessionDescription(sdp=offer_sdp, type="offer"))
    self.meta.addTrack(self.to_meta)
    await self.meta.setLocalDescription(await self.meta.createAnswer())
    await _gathered(self.meta)
    return self.meta.localDescription.sdp
```

AudioBridge.openai_offer · zovak/voice/webrtc_bridge.py

```
async def openai_offer(self) -> str:
    """Leg 2: our SDP offer for GPT-Live, made the way the website's browser does."""
    self.openai.addTrack(self.to_openai)
    self.openai.createDataChannel("oai-events")
    await self.openai.setLocalDescription(await self.openai.createOffer())
    await _gathered(self.openai)
    return self.openai.localDescription.sdp
```

16.3 Setup (Meta, free test number)

1. developers.facebook.com → My Apps → Create app → use case **Connect with customers through WhatsApp** (Business type) → link or create a business portfolio.
2. WhatsApp → API Setup: note the **test number**, its **Phone number ID**, and generate a **temporary access token** (24 h). Add your own WhatsApp number under **To** and verify the code.
3. App settings → Basic → **App secret**.
4. `.env`: `WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_APP_SECRET`, `WHATSAPP_VERIFY_TOKEN` (any long random string); restart uvicorn.
5. WhatsApp → Configuration → Webhook: callback `https://YOUR-NGROK/api/v1/whatsapp/webhook`, verify token = the same string; subscribe to **messages** (and **calls** for the call experiment).
6. **Subscribe your WhatsApp account to your app** — the dashboard's test flow subscribes it only to Meta's own test viewer ("WA DevX Webhook Events"), so messages appear in "Check test webhooks" but never reach ZOVAK: `curl -X POST https://graph.facebook.com/v25.0/WABA_ID/subscribed_apps -H "Authorization: Bearer $WA_TOKEN"`. Check with a GET on the same URL: your app must be listed.
7. Calls: enable calling on the number (`POST /PHONE_NUMBER_ID/settings` with `calling.status=ENABLED`) and subscribe the **calls** webhook field.

---

17. Doctors and staff

The demo clinic lives in `zovak/services/clinic.py` and is recreated on every start (`DEMO_SEED=true` loads demo bookings for today and tomorrow).

Doctor	Specialty	Services	Hours	Lunch
Dr. Ayesha Rahman	General Practice	General consultation, Follow-up visit	9 AM – 5 PM	1–2 PM
Dr. Imran Chowdhury	General Practice	General consultation, Follow-up visit	10 AM – 6 PM	2–3 PM
Dr. Nusrat Sultana	Pediatrics	Child check-up, Child vaccination	9 AM – 3 PM	12–1 PM
Dr. Farhan Karim	Dermatology	Skin consultation	11 AM – 7 PM	3–4 PM

Staff (demo login)	Role	PIN
Each doctor above	doctor — sees own bookings	1111 / 2222 / 3333 / 4444
Rina Akter	receptionist — sees everyone	5555
Tanvir Hasan	clinic manager — sees everyone	9999

17.1 Booking with a doctor

1. Nofa first asks what kind of doctor or problem the patient has (no diagnosis).
2. She calls `list_doctors` and matches the need to a specialty. If there's a match she calls `offer_doctors`: the website shows doctor cards (name, specialty, services, next free time; chat has a Choose button). On calls she says the service and only the matching doctors' names.
3. She never lists every doctor: `offer_doctors` refuses doctors from more than one specialty, and asked "which doctors do you have?" she describes the kinds of care (general, children's, skin) and asks about the problem first.
4. If nothing matches she says clearly that the clinic doesn't offer that service and lists what it does offer.
5. Availability, holds and overlaps are per doctor, so two GPs can see patients at the same time. If no doctor is named, the first free qualified doctor is assigned.
6. Google Calendar is one shared clinic calendar: ZOVAK subtracts its own bookings from Google's busy times (free/busy merges back-to-back events), so one doctor's booking doesn't block the others; outside events still block everyone.
7. Bookings keep the caller's phone/WhatsApp number (`customer_phone`) so patients can be told about delays later.
8. After a tap the card collapses to the picked time with a Change time link; the buttons only come back if the patient wants to change it.
9. Once a doctor is chosen, the card shows their free times as buttons (next 3 days with openings). In chat a tap sends the time as the patient's message; on a website call it goes to ZOVAK, which handles it like speech and Nofa answers aloud. A tap never books; the patient still confirms.
10. Picking a doctor (tapping Choose, typing or saying the name) is noticed by plain code before the AI runs: the cards are replaced by a "Doctor selected" card telling the patient to give a day and time, or ask for someone else. Naming two doctors or "not Rahman" isn't a pick, so the cards stay.

Doctor · zovak/services/clinic.py

```
@dataclass(frozen=True)
class Doctor:
    id: str
    name: str
    specialty: str
    services: tuple[str, ...]
    start: time
    end: time
    lunch: tuple[time, time]
    about: str = ""

    def works(self, start: datetime, end: datetime) -> bool:
        """Inside working hours and not over lunch (times in the clinic's timezone)."""
        a, b = start.time(), end.time()
        if start.date() != end.date() or a < self.start or b > self.end:
            return False
        return not (a < self.lunch[1] and b > self.lunch[0])
```

pick_offered_doctor · zovak/services/doctor_choice.py

```
def pick_offered_doctor(scheduler, tenant_id: str, conversation_id: str, text: str) -> str | None:
    """If the message names exactly one doctor, record the choice and return its id.

    Among the offered cards if there are any; otherwise any clinic doctor, so "book me
    with Dr. Farhan Karim" picks him straight away, with no cards first.
    """
    offered = getattr(scheduler, "offered_doctors", lambda *_: [])(tenant_id, conversation_id)
    booked = getattr(scheduler, "has_booking", lambda *_: False)(tenant_id, conversation_id)
    # After a booking, a doctor's name alone is talk about it; a new booking goes through cards.
    candidates = offered or ([] if booked else list(getattr(scheduler, "doctors", {})))
    if not candidates or _NOT_A_PICK.search(text):
        return None
    words = re.findall(r"[a-z]+", text.lower())
    matches = [d for d in candidates if _mentions(words, scheduler.doctors[d])]
    if len(matches) != 1:
        return None  # none or ambiguous: Nofa asks, the cards stay
    scheduler.choose_doctor(tenant_id, conversation_id, matches[0])
    return matches[0]
```

free_days · zovak/services/booking_state.py

```
async def free_days(scheduler, tenant_id: str, conversation_id: str, doctor_id: str,
                    *, days: int = 3, look_ahead: int = 10) -> list[dict]:
    """The chosen doctor's next days with free times, for tap-to-pick time buttons.

    Only truly free starts: inside their hours, not lunch, not booked, not past, and
    not held by another patient. Every service is 30 minutes, so the first one stands
    in for all of them.
    """
    from datetime import datetime, timedelta

    doctor = scheduler.doctors[doctor_id]
    today = datetime.now(scheduler.timezone).date()
    found = []
    for offset in range(look_ahead):
        day = today + timedelta(days=offset)
        try:
            slots = await scheduler.available_slots(
                tenant_id, doctor.services[0], day, doctor_id=doctor_id,
conversation_id=conversation_id,
            )
        except Exception:  # e.g. Google not connected: no buttons, typing still works
            return found
        if slots:
            found.append({"date": day.isoformat(), "times": [s.start.isoformat() for s in slots]})
            if len(found) == days:
                break
    return found
```

pick_tapped_time · zovak/services/doctor_choice.py

```
async def pick_tapped_time(scheduler, tenant_id: str, conversation_id: str, text: str) -> datetime
| None:
    """If the message is a tapped time for the chosen doctor and it's still free, remember it."""
    from zovak.services.booking_state import free_days

    found = _TAPPED_TIME.search(text)
    doctor_id = getattr(scheduler, "chosen_doctor", lambda *_: None)(tenant_id, conversation_id)
    if not found or doctor_id is None:
        return None
    for day in await free_days(scheduler, tenant_id, conversation_id, doctor_id):
        for iso in day["times"]:
            start = datetime.fromisoformat(iso)
            if f"{start:%B} {start.day}" == found[1] and start.strftime("%I:%M %p").lstrip("0") ==
found[2].upper():
                scheduler.pick_time(tenant_id, conversation_id, start)
                if getattr(scheduler, "moving_booking", lambda *_: None)(tenant_id,
conversation_id):
                    # Changing a confirmed booking: prepare the move now; they confirm or keep.
                    try:
                        await scheduler.propose_move(tenant_id, conversation_id, start)
                    except Exception:
                        return None
                return start
    return None
```

reopen_choice · zovak/services/doctor_choice.py

```
async def reopen_choice(scheduler, tenant_id: str, conversation_id: str, text: str, *,
                        emails: bool = True) -> str | None:
    """The patient wants to change what they picked: bring back the time buttons or doctor choice.

    Also while a proposal is waiting for confirmation: it is set aside (name and email kept)
    and the same doctor's time buttons come back.
    """
    if _TAPPED_TIME.search(text) or not hasattr(scheduler, "chosen_doctor"):
        return None  # a tapped time is a pick, not a change request
    if _NEVER_MIND.search(text):
        dropped = await scheduler.latest_pending(tenant_id, conversation_id)
        if dropped is not None:
            await scheduler.discard(tenant_id, conversation_id, dropped.id)  # nothing was booked
        change = await scheduler.latest_change(tenant_id, conversation_id)
        if change is not None:
            await scheduler.discard_change(tenant_id, conversation_id, change.id)  # booking
unchanged
        scheduler.drop_choices(tenant_id, conversation_id)
        return "dropped"
    if _NEW_TIME.search(text) and scheduler.has_booking(tenant_id, conversation_id) and not (
            await scheduler.latest_pending(tenant_id, conversation_id)):
        result = await scheduler.start_time_change(tenant_id, conversation_id)
        await note_for_agent(tenant_id, conversation_id, (
            "The patient wants another time for their confirmed booking. Its doctor's free times
are "
            "showing as buttons; when they tap one, the move is prepared for them to confirm."
            if result == "ok" else
            "The patient asked to change the time, but this booking's time was already changed
once. "
            "Tell them kindly that another change needs the clinic, and that they can still
cancel."))
        return "move" if result == "ok" else "move-used"
    if emails and wants_new_email(text):  # website calls use their own on-screen email box
        pending = await scheduler.latest_pending(tenant_id, conversation_id)
        if pending is not None and pending.customer_email:
            await scheduler.clear_proposal_email(tenant_id, conversation_id, pending.id)  # forgets
it too
            await note_for_agent(tenant_id, conversation_id,
                                 "The patient wants a different email, so the email box is showing
again. "
                                 "Ask them to type the new address in it; nothing else changes.")
            return "email"
        return None
    if _NEW_DOCTOR.search(text):
        if await scheduler.reopen_times(tenant_id, conversation_id) or
scheduler.chosen_doctor(tenant_id, conversation_id):
            scheduler.forget_doctor(tenant_id, conversation_id)
            return "doctor"
        return None
    if _NEW_TIME.search(text):
        if await scheduler.reopen_times(tenant_id, conversation_id):
            await note_for_agent(tenant_id, conversation_id,
                                 "The patient asked for a different time, so the unconfirmed
proposal was set "
                                 "aside. The time buttons are showing again; their name and email
are kept.")
            return "time"
        if scheduler.picked_time(tenant_id, conversation_id):
            scheduler.unpick_time(tenant_id, conversation_id)
            return "time"
    return None
```

wants_new_email · zovak/services/doctor_choice.py

```
def wants_new_email(text: str) -> bool:
    """The patient wants to change the email they typed (any channel)."""
    return bool(_NEW_EMAIL.search(text))
```

DemoScheduler.reopen_times · zovak/services/mock_scheduler.py

```
async def reopen_times(self, tenant_id: str, conversation_id: str) -> bool:
    """The patient wants another time while a proposal waits: set it aside, show the buttons.

    The doctor stays chosen and the name and email are kept for the next proposal.
    """
    pending = await self.latest_pending(tenant_id, conversation_id)
    if pending is None:
        return False
    key = (tenant_id, conversation_id)
    self._remember_contact(key, pending.customer_name, pending.customer_email)
    await self.discard(tenant_id, conversation_id, pending.id)
    if pending.doctor_id:
        self._chosen[key] = pending.doctor_id
    self._picked.pop(key, None)
    return True
```

DemoScheduler.start_time_change · zovak/services/mock_scheduler.py

```
async def start_time_change(self, tenant_id: str, conversation_id: str) -> str:
    """The patient wants another time for their confirmed booking: show its doctor's times.

    Returns "ok", "used" (already changed once) or "none" (no booking here).
    """
    booking = await self.latest_booking(tenant_id, conversation_id)
    if booking is None:
        return "none"
    if booking.time_changes >= MAX_TIME_CHANGES:
        return "used"
    key = (tenant_id, conversation_id)
    self._moving[key] = booking.id
    self._chosen[key] = booking.doctor_id
    self._picked.pop(key, None)
    return "ok"
```

stream_receptionist · zovak/agents/runner.py

```
async def stream_receptionist(scheduler, tenant_id: str, conversation_id: str, message: str, *,
                              channel: str = "chat") -> AsyncIterator[str]:
    """The same turn, yielding Nofa's words as the AI writes them (tools run first, unseen)."""
    from agents import Runner
    from openai.types.responses import ResponseTextDeltaEvent

    agent, context = await _prepare(scheduler, tenant_id, conversation_id, message, channel, None)
    session = memory_session(tenant_id, conversation_id)
    try:
        result = Runner.run_streamed(agent, message, context=context, session=session)
        async for event in result.stream_events():
            if event.type == "raw_response_event" and isinstance(event.data,
ResponseTextDeltaEvent):
                yield event.data.delta
    finally:
        await session.close()
```

VoiceSideband.pick_slot · zovak/voice/sideband.py

```
async def pick_slot(self, *, start: str, run_agent=None) -> dict:
    """A time button tapped on the website call: said for the caller, answered aloud.

    Only one of the chosen doctor's free times is accepted. It is handled exactly like
    the caller saying it, so it only proposes; confirming is still a separate step.
    """
    self.guard.caller_active()
    if self.closed:
        raise SchedulingError("The voice session has ended")
    doctor_id = self.scheduler.chosen_doctor(_TENANT, self.conversation_id)
    if doctor_id is None:
        raise SchedulingError("Choose a doctor first")
    try:
        moment = datetime.fromisoformat(start)
    except ValueError as exc:
        raise SchedulingError("That time isn't valid") from exc
    free = await free_days(self.scheduler, _TENANT, self.conversation_id, doctor_id)
    if moment.isoformat() not in {t for d in free for t in d["times"]}:
        raise SchedulingError("That time was just taken. Please pick another one.")
    text = slot_request(moment.astimezone(self.scheduler.timezone),
self.scheduler.doctors[doctor_id].name)
    logger.info("Caller tapped a time: %r", text)
    async with self._agent_lock:
        self._turn_heard_at = monotonic()
        reply = await self._handle_turn(text, run_agent or self._run_agent)
    await self._speak(reply)
    return {"message": reply}
```

function ChosenDoctor(· serena-health/src/components/assistant/AssistantWidget.tsx

```
function ChosenDoctor({
  doctor,
  voice,
  busy,
  onPick,
}: {
  doctor: ChosenDoctorInfo;
  voice: boolean;
  busy: boolean;
  onPick: (iso: string) => void;
}) {
  const [day, setDay] = useState(0);
  const [picked, setPicked] = useState<string | null>(null);
  // ZOVAK remembers the tapped time; the buttons only come back if the patient wants to change it.
  const [changing, setChanging] = useState(false);
  const pickerRef = useRef<HTMLDivElement>(null);
  // Reopened with "Change time": bring the buttons into view inside the chat.
  useEffect(() => {
    if (changing) pickerRef.current?.scrollIntoView({ block: "end" });
  }, [changing]);
  const days = doctor.slots;
  const times = days[Math.min(day, days.length - 1)]?.times ?? [];
  const chosen = picked ?? doctor.picked; // the tap shows at once; ZOVAK's copy after a reload
  // ZOVAK is the source of truth: when it records a time, or reopens the choice
  // ("change the time"), drop the local tap so the card follows it.
  useEffect(() => setPicked(null), [doctor.picked]);

  if (chosen && !changing) {
    return (
      <motion.div initial={{ opacity: 0, y: 8 }} animate={{ opacity: 1, y: 0 }} transition={{
duration: 0.3, ease }} className="w-full shrink-0">
        <CardShell tone="teal" title="Time selected" icon>
          <p className="font-bold">
            {tabDay.format(new Date(chosen))} · {timeFormat.format(new Date(chosen))}
          </p>
          <p className="text-ink-muted">with {doctor.name}</p>
          <div className="mt-2 flex items-center justify-between gap-3">
            <p className="text-[0.74rem] text-ink-faint">
              {busy ? "Sending…" : `Next: ${voice ? "say" : "tell"} ${site.agent} your name.
Nothing is booked yet.`}
            </p>
            <button
              type="button"
              onClick={() => {
                setPicked(null);
                setChanging(true);
              }}
… 74 more lines in the file
```

DemoScheduler.offer_doctors · zovak/services/mock_scheduler.py

```
def offer_doctors(self, tenant_id: str, conversation_id: str, doctor_ids: list[str]) -> list[str]:
    """Remember which doctors Nofa suggested, so the website can show them as cards."""
    known = [d for d in doctor_ids if d in self.doctors]
    if len({self.doctors[d].specialty for d in known}) > 1:
        # Never every doctor at once: only those matching the patient's need.
        raise SchedulingError("Offer doctors of one specialty only")
    if self._chosen.get((tenant_id, conversation_id)) in known:
        return known  # that doctor is already chosen: keep the choice, no cards again
    self._offers[(tenant_id, conversation_id)] = known
    self._chosen.pop((tenant_id, conversation_id), None)  # choosing again
    self._picked.pop((tenant_id, conversation_id), None)
    return known
```

DemoScheduler.available_slots · zovak/services/mock_scheduler.py

```
async def available_slots(self, tenant_id: str, service_id: str, day: date, *,
                          doctor_id: str | None = None,
                          ignore_booking_id: str | None = None,
                          conversation_id: str | None = None) -> list[Slot]:
    """Free 30-minute starts for a service, per doctor.

    doctor_id limits it to one doctor; otherwise each time is offered once with the
    first free qualified doctor. ignore_booking_id treats that booking's own time as
    free (moving it). conversation_id hides slots held by other conversations.
    """
    self._check_tenant(tenant_id)
    doctors = self.doctors_for(service_id, doctor_id)
    if day < datetime.now(self.timezone).date() or not doctors:
        return []
    duration = timedelta(minutes=self.services[service_id].duration_minutes)
    first = min(d.start for d in doctors)
    last = max(d.end for d in doctors)
    slots: list[Slot] = []
    now = datetime.now(timezone.utc)
    for minute in range(first.hour * 60 + first.minute, last.hour * 60 + last.minute, 30):
        local_start = datetime.combine(day, time(hour=minute // 60, minute=minute % 60),
self.timezone)
        local_end = local_start + duration
        if local_start.astimezone(timezone.utc) < now:
            continue
        for doctor in doctors:
            if (doctor.works(local_start, local_end)
                    and not self._overlaps(tenant_id, local_start, local_end,
                                           ignore_booking_id, doctor.id)
                    and not self._held_by_other(tenant_id, local_start, local_end,
                                                conversation_id, doctor.id)):
                slots.append(Slot(start=local_start, end=local_end, doctor_id=doctor.id))
                break
    return slots
```

GoogleBackedScheduler._external_busy · zovak/services/google_scheduler.py

```
def _external_busy(self, busy: list[tuple[datetime, datetime]]) -> list[tuple[datetime, datetime]]:
    """Busy time from events ZOVAK didn't create (meetings, holidays…).

    The clinic shares one Google calendar, so our own bookings show up as busy
    for every doctor. Free/busy has no event IDs and merges back-to-back events,
    so subtract our live bookings' intervals; what's left blocks all doctors.
    Each doctor's own bookings are checked locally, per doctor.
    """
    own = sorted((b.start, b.end) for b in self._bookings.values()
                 if b.status == "confirmed" and b.source == "live")
    external = []
    for start, end in busy:
        pieces = [(start, end)]
        for own_start, own_end in own:
            pieces = [piece for a, b in pieces for piece in (
                [(a, b)] if own_end <= a or own_start >= b
                else [p for p in ((a, own_start), (own_end, b)) if p[0] < p[1]]
            )]
        external.extend(pieces)
    return external
```

function DoctorCards(· serena-health/src/components/assistant/AssistantWidget.tsx

```
function DoctorCards({ doctors, onChoose }: { doctors: DoctorOption[]; onChoose?: (d: DoctorOption)
=> void }) {
  // On a call (no Choose button) a slim list that scrolls inside itself, so the
  // live transcript below always keeps its room.
  if (!onChoose) {
    return (
      <motion.div initial={{ opacity: 0, y: 8 }} animate={{ opacity: 1, y: 0 }} transition={{
duration: 0.3, ease }} className="w-full shrink-0">
        <p className="px-1 pb-1.5 text-[0.74rem] font-bold uppercase tracking-wider text-ink-
faint">
          Say the name of the doctor you&apos;d like
        </p>
        <ul data-lenis-prevent className="max-h-[9.5rem] divide-y divide-line overflow-y-auto
rounded-2xl border border-line bg-surface shadow-sm">
          {doctors.map((d) => (
            <li key={d.id} className="flex items-center gap-2.5 px-3 py-2">
              <span
                aria-hidden="true"
                className="flex h-8 w-8 shrink-0 items-center justify-center rounded-full bg-teal-
soft text-[0.75rem] font-bold text-teal"
              >
                {d.name.replace("Dr. ", "").split(" ").map((w) => w[0]).join("").slice(0, 2)}
              </span>
              <div className="min-w-0 flex-1">
                <p className="truncate text-[0.86rem] font-bold">
                  {d.name} <span className="font-normal text-ink-muted">· {d.specialty}</span>
                </p>
                <p className="truncate text-[0.74rem] text-ink-faint">
                  {d.nextFree ? `Next free: ${formatWhen(d.nextFree)}` : `Hours ${d.hours}`}
                </p>
              </div>
            </li>
          ))}
        </ul>
      </motion.div>
    );
  }
  return (
    <motion.div initial={{ opacity: 0, y: 8 }} animate={{ opacity: 1, y: 0 }} transition={{
duration: 0.3, ease }} className="w-full shrink-0 space-y-2">
      <p className="px-1 text-[0.74rem] font-bold uppercase tracking-wider text-ink-faint">
        Choose a doctor
      </p>
      {doctors.map((d) => (
        <div key={d.id} className="flex items-center gap-3 rounded-2xl border border-line bg-
surface p-3 shadow-sm">
          <span
            aria-hidden="true"
            className="flex h-11 w-11 shrink-0 items-center justify-center rounded-full bg-teal-
soft text-[0.95rem] font-bold text-teal"
          >
            {d.name.replace("Dr. ", "").split(" ").map((w) => w[0]).join("").slice(0, 2)}
          </span>
          <div className="min-w-0 flex-1">
            <p className="truncate text-[0.92rem] font-bold">{d.name}</p>
            <p className="truncate text-[0.8rem] text-ink-muted">
              {d.specialty} · {d.services.join(", ")}
            </p>
            <p className="text-[0.76rem] text-ink-faint">
              {d.nextFree ? `Next free: ${formatWhen(d.nextFree)}` : `Hours ${d.hours}`}
            </p>
          </div>
```

```
          <button
            type="button"
            onClick={() => onChoose(d)}
            className="min-h-[36px] shrink-0 cursor-pointer rounded-xl bg-teal px-3 text-[0.82rem]
font-bold text-on-accent hover:bg-teal-hover"
          >
            Choose
          </button>
        </div>
      ))}
    </motion.div>
  );
}
```

function ChosenDoctor(· serena-health/src/components/assistant/AssistantWidget.tsx

```
function ChosenDoctor({
  doctor,
  voice,
  busy,
  onPick,
}: {
  doctor: ChosenDoctorInfo;
  voice: boolean;
  busy: boolean;
  onPick: (iso: string) => void;
}) {
  const [day, setDay] = useState(0);
  const [picked, setPicked] = useState<string | null>(null);
  // ZOVAK remembers the tapped time; the buttons only come back if the patient wants to change it.
  const [changing, setChanging] = useState(false);
  const pickerRef = useRef<HTMLDivElement>(null);
  // Reopened with "Change time": bring the buttons into view inside the chat.
  useEffect(() => {
    if (changing) pickerRef.current?.scrollIntoView({ block: "end" });
  }, [changing]);
  const days = doctor.slots;
  const times = days[Math.min(day, days.length - 1)]?.times ?? [];
  const chosen = picked ?? doctor.picked; // the tap shows at once; ZOVAK's copy after a reload
  // ZOVAK is the source of truth: when it records a time, or reopens the choice
  // ("change the time"), drop the local tap so the card follows it.
  useEffect(() => setPicked(null), [doctor.picked]);

  if (chosen && !changing) {
    return (
      <motion.div initial={{ opacity: 0, y: 8 }} animate={{ opacity: 1, y: 0 }} transition={{
duration: 0.3, ease }} className="w-full shrink-0">
        <CardShell tone="teal" title="Time selected" icon>
          <p className="font-bold">
            {tabDay.format(new Date(chosen))} · {timeFormat.format(new Date(chosen))}
          </p>
          <p className="text-ink-muted">with {doctor.name}</p>
          <div className="mt-2 flex items-center justify-between gap-3">
            <p className="text-[0.74rem] text-ink-faint">
              {busy ? "Sending…" : `Next: ${voice ? "say" : "tell"} ${site.agent} your name.
Nothing is booked yet.`}
            </p>
            <button
              type="button"
              onClick={() => {
                setPicked(null);
                setChanging(true);
              }}
              className="shrink-0 cursor-pointer text-[0.8rem] font-bold text-teal underline-
offset-2 hover:underline"
            >
              Change time
            </button>
          </div>
        </CardShell>
      </motion.div>
    );
  }

  return (
    <motion.div
      ref={pickerRef}
      initial={{ opacity: 0, y: 8 }}
```

```
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.3, ease }}
      className="w-full shrink-0"
    >
      <CardShell tone="teal" title={doctor.purpose === "move" ? "Pick your new time" : "Doctor
selected"} icon>
        <p className="font-bold">{doctor.name}</p>
        <p className="text-ink-muted">
          {doctor.specialty} · Hours {doctor.hours}
        </p>
        {days.length > 0 ? (
          <>
            <p className="mt-3 text-[0.76rem] font-bold uppercase tracking-wider text-ink-
faint">Pick a free time · Dhaka</p>
            <div role="tablist" aria-label="Day" className="mt-1.5 flex gap-1.5 overflow-x-auto pb-
1">
              {days.map((d, i) => (
                <button
                  key={d.date}
                  role="tab"
                  aria-selected={i === day}
                  onClick={() => setDay(i)}
                  className={`shrink-0 cursor-pointer rounded-full px-3 py-1 text-[0.78rem] font-
bold transition-colors ${
                    i === day ? "bg-teal text-on-accent" : "border border-line-strong text-ink-
muted hover:bg-canvas-soft"
                  }`}
                >
                  {tabDay.format(new Date(d.date + "T12:00:00+06:00"))}
                </button>
              ))}
            </div>
            <div data-lenis-prevent className="mt-2 grid max-h-[7.5rem] grid-cols-4 gap-1.5
overflow-y-auto pr-0.5">
              {times.map((iso) => (
                <button
                  key={iso}
                  disabled={busy}
                  onClick={() => {
                    setPicked(iso);
                    setChanging(false);
                    onPick(iso);
                  }}
                  className={`min-h-[34px] cursor-pointer rounded-lg border text-[0.8rem] font-bold
tabular-nums transition-colors disabled:cursor-wait ${
                    (picked ?? doctor.picked) === iso ? "border-teal bg-teal text-on-accent" :
"border-teal/25 text-teal hover:bg-teal-soft"
                  }`}
                >
                  {timeFormat.format(new Date(iso)).replace(/\s?(AM|PM)/, (m) =>
m.trim().toLowerCase())}
                </button>
              ))}
            </div>
            <p className="mt-2 text-[0.74rem] text-ink-faint">
              {doctor.purpose === "move"
                ? "Your one time change: tap a new time, then confirm the move (or keep your
current time)."
                : `Tap a time${voice ? ", or say one" : " or type one"}. Nothing is booked until
you confirm. Want someone else? Just ${voice ? "say so" : "ask"}.`}
            </p>
          </>
        ) : (
          <p className="mt-2 text-[0.76rem] text-ink-faint">
```

```
            Next: tell {site.agent} the day and time you&apos;d like. Want someone else? Just
{voice ? "say so" : "ask"}.
          </p>
        )}
      </CardShell>
    </motion.div>
  );
}
```

18. Delay recovery and the staff portal

18.1 Only the patients the delay reaches

When the delay starts counting: a time staff give ("30 minutes late for my 1:00"); otherwise now while the doctor is working, their start time if their day hasn't begun, or their next day's start if today is over. A report at 10 PM therefore applies to tomorrow's patients.

Report	Counts from	Result for Dr. Karim (11:00–7:00; patients 11:30, 12:00, 12:30, 1:00)
10 PM: "30 minutes late"	tomorrow 11:00	Nobody: the delay ends at 11:30, in time for the 11:30 patient
10 PM: "30 minutes late for my 1:00"	tomorrow 1:00	Only the 1:00 patient → 1:30
11:15 AM (working): "30 minutes late"	11:15	11:30 → 11:45, 12:00 → 12:15, 12:30 → 12:45, 1:00 → 1:15

Each of the doctor's remaining visits that day starts at *max(its own time, end of the previous shifted visit)*, skipping lunch. As soon as a visit already starts after the doctor would be free, the gap has absorbed the delay and nothing later changes.

Dr. Rahman +60 min at 9:50	Before	After	Why
Ali	10:00	10:50	affected
Bina	10:30	11:20	affected
Chandni	11:00	11:50	affected
Dipu	11:30	12:20	affected
Eshita	2:00 PM	2:00 PM	untouched: the gap before lunch absorbed the delay

18.2 Options per patient (best first)

1. **No wait:** the same original time with another doctor who offers the service and is free (never offering one slot twice).
2. **New time** with the same doctor: the shifted time. This is the default.
3. **Other times** with the same doctor, only after they arrive: today, then tomorrow, avoiding the shifted visits.

4. **Cancel.** If the shifted visit would run past closing, the default becomes the first next free time ("needs a new time").

18.3 Preview, send, patient choice

1. Staff (or the doctor) choose the delay and, optionally, *Late from* a specific day and time, then press *See who is affected*: a preview saying when it counts from; nothing changes and nobody is notified. If nobody is affected it says why and there's nothing to send.
2. *Notify patients* applies every default time, latest first so shifts never collide, then sends each patient a message with a one-time link: SMS router (test inbox / ntfy / provider), WhatsApp if they booked there, and email if Resend is set up. Nofa's memory gets a note too.
3. The patient opens `/update/<token>` and taps an option. Another doctor or another time moves the booking (Google Calendar is updated for live bookings); cancel cancels it. The link works once.
4. Staff watch *Patient updates*: waiting, kept, rescheduled, cancelled, needs staff, or no reply after 15 minutes (they keep the new time).

RecoveryPlanner.delay_start · zovak/services/recovery.py

```
def delay_start(self, doctor, now: datetime, late_from: datetime | None = None) -> datetime:
    """When the delay begins: a given time, else now while working, else the next start."""
    tz = self.scheduler.timezone
    if late_from is not None:
        late_from = (late_from if late_from.tzinfo else
late_from.replace(tzinfo=tz)).astimezone(tz)
        if late_from < now - timedelta(hours=1):
            raise SchedulingError("That time has already passed")
        return late_from
    start = datetime.combine(now.date(), doctor.start, tz)
    end = datetime.combine(now.date(), doctor.end, tz)
    if now < start:
        return start  # before their day: late for the start of it
    if now < end:
        return now
    return start + timedelta(days=1)  # their day is over: the next one
```

RecoveryPlanner.preview · zovak/services/recovery.py

```
async def preview(self, doctor_id: str, delay_minutes: int, reported_by: str, *,
                  now: datetime | None = None, late_from: datetime | None = None) -> DelayPlan:
    scheduler = self.scheduler
    doctor = scheduler.doctors.get(doctor_id)
    if doctor is None:
        raise SchedulingError("Unknown doctor")
    if not 5 <= delay_minutes <= 240:
        raise SchedulingError("Delay must be between 5 and 240 minutes")
    now = (now or datetime.now(timezone.utc)).astimezone(scheduler.timezone)
    base = self.delay_start(doctor, now, late_from)
    todays = sorted(
        (b for b in scheduler._bookings.values()
         if b.doctor_id == doctor_id and b.status == "confirmed"
         and b.start.astimezone(scheduler.timezone).date() == base.date() and b.end > base),
        key=lambda b: b.start,
    )
    # Pass 1: ripple the delay forward until a gap absorbs it.
    cursor = base + timedelta(minutes=delay_minutes)
    shifted: list[tuple] = []
    for booking in todays:
        if booking.start >= cursor:
            break  # this one and everything after it still fits: untouched
        duration = booking.end - booking.start
        new_start = _fit(doctor, max(booking.start, cursor).astimezone(scheduler.timezone),
duration)
        shifted.append((booking, new_start))
        if new_start is not None:
            cursor = new_start + duration
    reserved = [(s, s + (b.end - b.start)) for b, s in shifted if s is not None]

    # Pass 2: options for each affected patient.
    items = []
    taken: set[tuple[str, datetime]] = set()  # other-doctor slots offered in this plan
    for booking, new_start in shifted:
        options: list[Option] = []
        if booking.start > now:
            for other in scheduler.doctors.values():
                if (other.id != doctor_id and booking.service_id in other.services
                        and (other.id, booking.start) not in taken
                        and other.works(booking.start.astimezone(scheduler.timezone),
                                        booking.end.astimezone(scheduler.timezone))
                        and not scheduler._overlaps(TENANT, booking.start, booking.end,
                                                    doctor_id=other.id)):
                    taken.add((other.id, booking.start))
                    options.append(Option("other", "other_doctor",
                                          f"{_t(booking.start, scheduler)} with {other.name} (no
wait)",
                                          booking.start, other.id))
                    break
        if new_start is not None:
            options.append(Option("shifted", "shifted",
                                  f"{_t(new_start, scheduler)} with {doctor.name} (new time)",
                                  new_start, doctor_id))
        arrives = base + timedelta(minutes=delay_minutes)  # nothing earlier with this doctor
        for slot in await self._next_free(booking, reserved, arrives, exclude=new_start):
            options.append(Option(f"slot{len(options)}", "next_slot",
                                  f"{_day_t(slot.start, now, scheduler)} with {doctor.name}",
                                  slot.start, doctor_id))
        options.append(Option("cancel", "cancel", "Cancel my appointment"))
        default = "shifted" if new_start else next((o.id for o in options if o.kind ==
"next_slot"), None)
        items.append(PlanItem(
```

```
            booking_id=booking.id, conversation_id=booking.conversation_id,
            patient=booking.customer_name, phone=booking.customer_phone,
            email=booking.customer_email, service_id=booking.service_id,
            original_start=booking.start, shifted_start=new_start,
            options=options, default_option=default,
        ))
    plan = DelayPlan(id=uuid4().hex[:10], doctor_id=doctor_id, delay_minutes=delay_minutes,
                     reported_by=reported_by, created_at=datetime.now(timezone.utc), items=items,
                     starts_at=base)
    if not items:
        plan.note = self._why_nobody(doctor, base, delay_minutes, todays, now)
… 2 more lines in the file
```

RecoveryPlanner._why_nobody · zovak/services/recovery.py

```
def _why_nobody(self, doctor, base: datetime, minutes: int, bookings, now: datetime) -> str:
    """Plain words for staff: why this delay doesn't reach anyone."""
    s = self.scheduler
    day = _day_name(base, now)
    free_at = base + timedelta(minutes=minutes)
    if not bookings:
        return (f"{doctor.name} has no patients {day} from {_t(base, s)}, "
                "so nobody needs to be told.")
    first = bookings[0]
    return (f"{doctor.name}'s first patient {day} from {_t(base, s)} is {first.customer_name} at "
            f"{_t(first.start, s)}. A {minutes}-minute delay ends at {_t(free_at, s)}, in time for
"
            "them, so nobody needs to be told.")
```

RecoveryPlanner.send · zovak/services/recovery.py

```
async def send(self, plan_id: str) -> DelayPlan:
    plan = self.plans.get(plan_id)
    if plan is None or plan.status != "draft":
        raise SchedulingError("This plan was already sent or discarded")
    doctor = self.scheduler.doctors[plan.doctor_id]
    # Latest first, so each shift moves into time the next one already freed.
    for item in sorted(plan.items, key=lambda i: i.original_start, reverse=True):
        option = next((o for o in item.options if o.id == item.default_option), None)
        if option is None or option.start is None:
            item.status = "needs_staff"
            continue
        try:
            await self.scheduler.staff_move(TENANT, item.booking_id, option.start,
                                            doctor_id=option.doctor_id)
            item.status = "notified"
        except SchedulingError as exc:
            logger.warning("Could not shift booking %s: %s", item.booking_id, exc)
            item.status = "needs_staff"
    plan.status, plan.sent_at = "sent", datetime.now(timezone.utc)
    for index, item in enumerate(plan.items):
        self._tokens[item.token] = (plan.id, index)
        await self._notify(plan, item, doctor)
    return plan
```

RecoveryPlanner.choose · zovak/services/recovery.py

```
async def choose(self, token: str, option_id: str) -> PlanItem:
    found = self.lookup(token)
    if found is None:
        raise SchedulingError("This link is not valid")
    plan, item = found
    if item.choice is not None:
        raise SchedulingError("You already chose an option")
    option = next((o for o in item.options if o.id == option_id), None)
    if option is None:
        raise SchedulingError("That option is no longer available")
    if option.kind == "cancel":
        await self.scheduler.staff_cancel(TENANT, item.booking_id)
        item.status = "cancelled"
    elif option.id == item.default_option and item.status == "notified":
        item.status = "kept"
    else:
        await self.scheduler.staff_move(TENANT, item.booking_id, option.start,
doctor_id=option.doctor_id)
        item.status = "rescheduled"
    item.choice = option.id
    await note_for_agent(TENANT, item.conversation_id,
                         f"After the doctor's delay the patient chose: {option.label}.")
    return item
```

DemoScheduler.staff_move · zovak/services/mock_scheduler.py

```
async def staff_move(self, tenant_id: str, booking_id: str, new_start: datetime, *,
                     doctor_id: str | None = None) -> BookingProposal:
    """Move a confirmed booking (optionally to another doctor). Updates Google for live
bookings."""
    self._check_tenant(tenant_id)
    async with self._lock:
        booking = self._bookings.get(booking_id)
        if booking is None or booking.status != "confirmed":
            raise SchedulingError("Booking not found")
        target = doctor_id or booking.doctor_id
        doctor = self.doctors.get(target or "")
        new_start = new_start.astimezone(self.timezone)
        new_end = new_start + (booking.end - booking.start)
        if doctor is not None:
            if booking.service_id not in doctor.services:
                raise SchedulingError(f"{doctor.name} doesn't offer this service")
            if not doctor.works(new_start, new_end):
                raise SchedulingError(f"Outside {doctor.name}'s hours")
        if self._overlaps(tenant_id, new_start, new_end, ignore_booking_id=booking.id,
                          doctor_id=target):
            raise SchedulingError("That time is already booked")
        change = ChangeRequest(
            id=str(uuid4()), tenant_id=tenant_id, conversation_id=booking.conversation_id,
            booking_id=booking.id, kind="move", new_start=new_start, new_end=new_end,
            expires_at=datetime.now(timezone.utc) + timedelta(minutes=5),
        )
        await self._calendar_apply(change, booking)
        booking.start, booking.end, booking.doctor_id = new_start, new_end, target
        booking.confirmed_at = datetime.now(timezone.utc)
        booking.reminder_sent = False
        return booking
```

18.4 Staff portal (/staff on the website)

Where the doctor's schedule comes from: every booking, whether made in website chat, a website call, a phone call or the SMS form, is stored once in ZOVAK's booking store with its doctor. The portal reads that same store, so a new booking shows in that doctor's login within 5 seconds. Doctors have no calendars of their own; the one shared clinic Google Calendar only sends patients their invites. The store is in memory for now (lost on restart) until saved bookings are planned.

Tab	What it does
Sign in	Name + demo PIN → session token (12 h; 5 wrong PINs lock that name for 5 minutes)
Schedule	Today, Tomorrow, or any day (‹ › or the date picker), per doctor; past visits greyed; delay status badges; refreshes every 5 s. Doctors see only themselves
Running late	Pick doctor (reception/manager), minutes and Late from (Now / a specific time) → preview → Notify patients or Discard
Patient updates	Every sent plan with each patient's status and choice, refreshed every few seconds
Assistant	Staff chat with Nofa: schedules, and "I'm 40 minutes late" or "late for my 1:00" → preview → explicit "send it". Replies are written out word by word and the chat keeps the newest message in view. Each sign-in has its own memory; the chat survives a reload and is cleared on sign-in/sign-out

delay_preview · zovak/api/routes/staff.py

```
@router.post("/delay/preview")
async def delay_preview(body: DelayRequest, request: Request,
                        x_staff_token: str | None = Header(default=None)) -> dict:
    member = _staff(request, x_staff_token)
    doctor_id = member.doctor_id if member.role == "doctor" else body.doctor_id
    if doctor_id not in _visible_doctors(request, member):
        raise HTTPException(403, detail="You can only report delays for your own schedule")
    try:
        plan = await request.app.state.recovery.preview(doctor_id, body.minutes, member.name,
                                                        late_from=body.late_from)
    except SchedulingError as exc:
        raise HTTPException(409, detail=str(exc)) from exc
    return request.app.state.recovery.view(plan)
```

make_staff_agent · zovak/agents/staff.py

```
def make_staff_agent(staff) -> Agent[StaffContext]:
    who = (f"You are speaking with {staff.name}, a doctor; act only on their own schedule."
           if staff.role == "doctor" else
           f"You are speaking with {staff.name} ({staff.role}); they can manage every doctor.")
    return Agent[StaffContext](
        name="Nofa Staff Assistant",
        model=get_settings().openai_agent_model,
        instructions=(
            "You are Nofa's assistant for clinic staff. " + who + " " + today_instructions() +
            "Answer schedule questions with the schedule tool. When someone is running late, "
            "call preview_delay every time, even if you planned a delay earlier in this chat "
            "(schedules change; never reuse an old result). If they name a time (\"30 minutes late
for my 1:00\"), pass "
            "late_from with that appointment's date and time; if they say when they'll arrive "
            "(\"I won't be in until 2\"), pass late_from = the first appointment they'll miss and "
            "minutes = the gap to their arrival. For \"running late\" now, leave late_from out. "
            "Then summarize who is affected and their options, and ask for "
            "confirmation. Only call send_delay_plan after the staff member clearly confirms "
            "(e.g. 'yes, send it'). Be brief and answer only what was asked: don't bring up earlier
"
            "delay plans unless the staff member asks about them. Patients' names and phone numbers
are private: "
            "only share them with this logged-in staff member."
        ),
        tools=[schedule, preview_delay, send_delay_plan],
    )
```

run_staff_agent · zovak/agents/staff.py

```
async def run_staff_agent(staff, scheduler, planner, message: str, *, chat_id: str = "default") ->
str:
    from agents import Runner

    from zovak.agents.memory import memory_session
    # One memory per sign-in: a restart or new sign-in starts a clean conversation.
    session = memory_session("demo-business", f"staff-{staff.id}-{chat_id}")
    try:
        result = await Runner.run(make_staff_agent(staff), message,
                                  context=StaffContext(staff, scheduler, planner), session=session)
        return str(result.final_output)
    finally:
        await session.close()
```

StaffSessions.login · zovak/services/staff_auth.py

```
def login(self, staff_id: str, pin: str) -> tuple[str, StaffMember] | None:
    member = STAFF.get(staff_id)
    now = time.monotonic()
    recent = [t for t in self._failures.get(staff_id, []) if now - t < 300]
    if member is None or len(recent) >= MAX_ATTEMPTS:
        return None
    if not compare_digest(member.pin, pin):
        self._failures[staff_id] = recent + [now]
        return None
    token = token_urlsafe(32)
    self._sessions[token] = (staff_id, now + SESSION_HOURS * 3600)
    self._chats[token] = token_hex(8)
    return token, member
```

function RunningLate(· serena-health/src/app/staff/page.tsx

```
function RunningLate({ call, me, onSent }: { call: Call; me: Me; onSent: () => void }) {
  const [doctors, setDoctors] = useState<DoctorDay[]>([]);
  const [doctorId, setDoctorId] = useState(me.doctor_id ?? "");
  const [minutes, setMinutes] = useState(30);
  // "Now" lets the backend pick the moment (now, or the doctor's next start time);
  // a specific time is for "30 minutes late for my 1:00".
  const [fromWhen, setFromWhen] = useState<"now" | "time">("now");
  const [lateDate, setLateDate] = useState(dhakaDate(0));
  const [lateTime, setLateTime] = useState("13:00");
  const [plan, setPlan] = useState<Plan | null>(null);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (me.role === "doctor") return;
    call<{ doctors: DoctorDay[] }>(`schedule?day=${dhakaDate(0)}`).then((d) => {
      setDoctors(d.doctors);
      setDoctorId((current) => current || d.doctors[0]?.id || "");
    });
  }, [call, me.role]);

  async function run<T>(work: () => Promise<T>) {
    setBusy(true);
    setError(null);
    try {
      return await work();
    } catch (err) {
      setError((err as Error).message);
    } finally {
      setBusy(false);
    }
  }

  const preview = () =>
    run(async () => {
      const lateFrom = fromWhen === "time" ? `${lateDate}T${lateTime}` : undefined;
      setPlan(await call<Plan>("delay/preview", { method: "POST", body: { doctor_id: doctorId,
minutes, late_from: lateFrom } }));
    });
  const send = () =>
    run(async () => {
… 179 more lines in the file
```

19. The widget UI

- **Font:** Figtree for the widget (`src/app/layout.tsx`, Tailwind `font-widget`).
- **Chat:** Nofa's bubbles are white with a small orb avatar; the customer's are teal; replies stream in word by word; the list keeps scrolled to the newest message unless you scroll up.
- **Voice:** before content appears, a large orb; during a call with transcript or a card, a slim header with a small orb, status and a live timer. The card never shrinks; the transcript scrolls in the remaining space. Doctor suggestions on a call are a slim list (name, specialty, next free time) that scrolls inside itself, so the transcript always stays visible.
- **Greeting:** "Hi, I'm Nofa, the front desk for Brightside Family Clinic. Tell me what you need, and I'll find the right doctor and a time that suits you. I can also move or cancel a booking." Buttons: I need

to see a doctor · What services do you offer? · Change or cancel my booking.

function useCallTimer(· serena-health/src/components/assistant/AssistantWidget.tsx

```
function useCallTimer(state: VoiceState): number {
  const [startedAt, setStartedAt] = useState<number | null>(null);
  const [elapsed, setElapsed] = useState(0);
  const live = state === "listening" || state === "thinking" || state === "speaking";

  useEffect(() => {
    if (state === "connecting") {
      setStartedAt(null);
      setElapsed(0);
    } else if (live && startedAt === null) {
      setStartedAt(Date.now());
    } else if (state === "idle") {
      setStartedAt(null);
    }
  }, [state, live, startedAt]);

  useEffect(() => {
    if (startedAt === null) return;
    const tick = () => setElapsed(Math.floor((Date.now() - startedAt) / 1000));
    tick();
    const id = setInterval(tick, 1000);
    return () => clearInterval(id);
  }, [startedAt]);

  return elapsed;
}
```

function BookingPanel(· serena-health/src/components/assistant/AssistantWidget.tsx

```
function BookingPanel(props: Parameters<typeof BookingPanelBody>[0] & { onDismiss?: () => void }) {
  const { onDismiss, ...rest } = props;
  const s = props.state;
  if (s.stage === "idle" && s.doctorOptions.length === 0 && !s.chosenDoctor) return null;
  return (
    <div className="relative w-full">
      {onDismiss && (
        <button
          type="button"
          onClick={onDismiss}
          aria-label="Hide this card"
          title="Hide this card"
          className="absolute -right-1.5 -top-1.5 z-10 flex h-6 w-6 cursor-pointer items-center
justify-center rounded-full border border-line-strong bg-surface text-[0.7rem] text-ink-muted
shadow-sm hover:text-ink"
        >
          ×
        </button>
      )}
      <BookingPanelBody {...rest} />
    </div>
  );
}
```

20. File map

File	What it does
`zovak/agents/receptionist.py`	Agent instructions and tools: propose booking/move/cancel, current booking, report off-topic, today's date
`zovak/services/mock_scheduler.py`	Booking rules in memory: slots, proposals, confirm, move/cancel
`zovak/services/google_scheduler.py`	Same rules plus Google Calendar free/busy, create, move, delete
`zovak/services/booking_state.py`	Decides the card's stage, and the live status text given to the agent
`zovak/agents/memory.py`	Agent memory per conversation, plus notes about button actions
`zovak/providers/calendar/google_invites.py`	Google Calendar REST calls
`zovak/voice/sideband.py`	Live call brain: delegation, card actions, spoken confirmations, call ending
`zovak/voice/confirmation.py`	Spoken yes/no rules: natural but never a possibly-unfinished yes
`zovak/voice/call_guard.py`	Silence, off-topic and too-many-changes limits
`zovak/api/routes/phone.py`	Phone webhook: verify, accept SIP calls, attach the sideband
`zovak/api/routes/forms.py`	Public one-time SMS booking form pages
`zovak/services/booking_forms.py`	Form tokens, 2-minute holds, submit (confirm/cancel)
`zovak/providers/notifications/sms_outbox.py`	Development SMS test inbox
`zovak/providers/notifications/sms_router.py`	Sends each text to the test inbox, ntfy and the right provider by country
`zovak/providers/notifications/ntfy_push.py`	Free push-notification stand-in for SMS (testing)
`zovak/providers/notifications/telnyx_sms.py`	Telnyx SMS adapter
`zovak/providers/notifications/whatsapp_meta.py`	WhatsApp Cloud API client: text, buttons, read receipts, call actions
`zovak/services/whatsapp_bot.py`	WhatsApp chatbot with tap-button booking card

File	What it does
`zovak/services/whatsapp_calls.py`	WhatsApp calls: Meta webhook → audio bridge → GPT-Live + sideband
`zovak/voice/webrtc_bridge.py`	Audio bridge (aiortc) between two ICE-lite WebRTC servers
`zovak/api/routes/whatsapp.py`	Public WhatsApp webhook: verification and signed events
`zovak/agents/runner.py`	Runs one agent turn for text channels (website chat, WhatsApp)
`zovak/services/clinic.py`	Demo doctors, services, staff logins and demo bookings
`zovak/services/recovery.py`	Delay recovery: ripple planner, options, send, patient choice
`zovak/services/staff_auth.py`	Demo staff sign-in sessions
`zovak/api/routes/staff.py`	Staff portal API (local only)
`zovak/agents/staff.py`	Staff assistant agent
`serena-health/src/app/staff/page.tsx`	The staff portal page
`zovak/voice/email_form.py`	Typed email validation and gate
`zovak/api/routes/dev.py`	Chat routes: chat, confirm, booking-state, email, change
`zovak/api/routes/voice_dev.py`	Voice routes: session, booking-state, card actions (token required)
`zovak/workers/reminders.py`	1-hour reminder job
`zovak/providers/notifications/resend_email.py`	Resend email sender
`serena-health/src/components/assistant/AssistantWidget.tsx`	The widget: chat, voice, booking card, timer
`serena-health/src/lib/agent/client.ts`	Browser side: chat, voice WebRTC, card polling and actions
`serena-health/src/app/api/agent/*`	Next.js routes that forward to ZOVAK

21. Limits and what's next

- **Memory only:** bookings, reminders and Google sign-in are lost when uvicorn restarts. Move/cancel only works in the conversation where the booking was made (WhatsApp chat and WhatsApp calls from one number share a conversation; phone and website don't link to it yet). A planned customer-tracking system with saved bookings will fix this.

- **Development routes have no login** and must not be exposed publicly.
- **Voice timing is estimated:** there is no guaranteed "caller finished" signal, and the goodbye wait is estimated from its length.
- **Phone:** Linphone calls are not yet tested against OpenAI end to end; a real number needs a SIP provider with TLS + SRTP to OpenAI. Caller ID is untrusted, so calling back to change a booking will need saved bookings plus a one-time SMS code.
- **SMS:** carriers reject texts from unregistered sender names (Sweden: 40008; Bangladesh: 40305/40306), so real SMS needs a registered sender or a local gateway per country; links may also be filtered. Bangladesh needs a local gateway or a registered sender; India needs DLT registration.

---

22. Why it's built this way (decision record)

Every connection in this system was tested in practice. This section keeps the reasons, so nobody has to rediscover them: what was tried, what happened, the evidence, and the code that implements the choice.

22.1 Website ↔ ZOVAK

Tried	Result	Why / evidence
Browser calls ZOVAK directly	❌ **Not used**	ZOVAK has no CORS or login; exposing it would leak control of bookings and the OpenAI key
Next.js server routes forward to ZOVAK	✅ **Chosen**	Browser only talks to its own site; ZOVAK stays on 127.0.0.1

Chosen: The site's /api/agent/* routes call ZOVAK server-side with zovakFetch.

Code:

- `serena-health/src/lib/agent/zovak.ts → zovakFetch`
- `serena-health/src/app/api/agent/{chat,session,booking}/route.ts`

22.2 Confirming bookings

Tried	Result	Why / evidence
Let the AI book directly	❌ **Rejected**	An LLM can misread or be tricked; safety rule: the AI proposes, the customer confirms
Strict spoken phrases only	❌ **Too rigid**	Customers had to say exact sentences; filler words broke it
AI-classified "yes" (outside package)	❌ **Rejected**	Didn't install on our code, broke 12 tests, could misread "yes, but…"
Buttons + natural-but-safe spoken yes + 1.5 s silence check	✅ **Chosen**	An action word is required, hedge words block it, and a yes followed by more speech is ignored

Chosen: Tap/click buttons everywhere; spoken yes only when complete and unconditional; phone confirmation via the SMS form.

Code:

- `zovak/voice/confirmation.py → is_approval, classify_confirmation`
- `zovak/voice/sideband.py → _settled, _commit_booking`
- `zovak/services/booking_state.py → booking_state`
- `serena-health AssistantWidget.tsx → BookingPanel`

22.3 Phone calls

Tried	Result	Why / evidence
Linphone app → OpenAI SIP directly	❌ **No audio**	Call set up and ZOVAK accepted, but no audio packets ever flowed: red signal in Linphone, no "Nofa is speaking", nothing billed, closed after ~30 s. A phone behind a router/mobile network can't receive OpenAI's audio
Twilio trial	❌ **Not available**	No free SIP trunking on the trial
Telnyx trial	❌ **Blocked**	Trial only sells a number in the account's home country (Bangladesh)
Telnyx paid: US local number → FQDN SIP to OpenAI (TLS + SRTP)	✅ **Works**	Carrier servers carry the audio; ZOVAK code unchanged. A real Bangladeshi number later uses the same path

Chosen: A SIP provider number routed to sip:proj_…@sip.api.openai.com; OpenAI's signed webhook; ZOVAK accepts and attaches the sideband.

Code:

- `zovak/api/routes/phone.py → openai_webhook, caller_id`
- `zovak/voice/sideband.py → _listen, _phone_turn, _send_form`
- `zovak/main.py → _guard_public_host`

22.4 Sending the booking form (SMS)

Tried	Result	Why / evidence
Telnyx SMS → Bangladesh	❌ **Rejected**	40305 (sender name not registered) / 40306 (international needs an alpha sender)
Telnyx SMS → Sweden	❌ **Undelivered**	Accepted (queued) then 40008 "carrier did not accept"
ntfy push (testing)	✅ **Chosen for testing**	Free, instant; tapping opens the form
Local BD gateway (e.g. Alpha SMS)	⏳ **Planned**	Cheap, registered/non-masking senders accepted by BD carriers

Chosen: Every text goes through SmsRouter: always the test inbox, ntfy when set, a real provider chosen by country code. Adding a provider never touches call or form code.

Code:

- `zovak/providers/notifications/sms_router.py → SmsRouter`
- `zovak/providers/notifications/ntfy_push.py`

- `zovak/providers/notifications/telnyx_sms.py`
- `zovak/services/booking_forms.py`
- `zovak/api/routes/forms.py`

22.5 WhatsApp chat

Tried	Result	Why / evidence
Messages reached Meta but not ZOVAK	❌ **First attempt**	The test flow had subscribed the WhatsApp account only to Meta's own viewer app ("WA DevX Webhook Events"), so payloads showed in "Check test webhooks" only
POST /{WABA_ID}/subscribed_apps with our app's token	✅ **Fixed**	Our app appears in subscribed_apps; messages arrive signed at /api/v1/whatsapp/webhook

Chosen: Meta Cloud API test number (free, ≤5 recipients); same agent as website chat; tap buttons as the booking card; typing indicator.

Code:

- `zovak/services/whatsapp_bot.py → WhatsAppBot`
- `zovak/api/routes/whatsapp.py → verify, receive`
- `zovak/providers/notifications/whatsapp_meta.py → WhatsAppCloud`

22.6 WhatsApp calls

Tried	Result	Why / evidence
Meta SIP → OpenAI directly	❌ **Can't work**	Meta puts the business number in the Request-URI (OpenAI needs proj_…), expects a digest challenge and the first SRTP packet from our side
Telnyx WhatsApp calling	❌ **Blocked**	Meta requires a 2,000-recipient messaging limit before calling (new accounts start lower)
Hand Meta's WebRTC offer straight to GPT-Live	❌ **No audio**	Logged SDPs: Meta a=ice-lite + setup:actpass, GPT-Live a=ice-lite + setup:passive. Both only answer connection checks, neither starts them
ZOVAK audio bridge (aiortc): full WebRTC client on both legs	✅ **Works**	Verified locally with two real peers, then a live WhatsApp call answered by Nofa

Chosen: ZOVAK answers Meta's offer and calls GPT-Live like a browser, relaying audio (mono 48 kHz, real-time pacing, silence until Nofa speaks).

Code:

- `zovak/voice/webrtc_bridge.py → AudioBridge, RelayTrack`
- `zovak/services/whatsapp_calls.py → WhatsAppCalls._connect, sdp_summary`

22.7 The agent's memory

Tried	Result	Why / evidence
Agent memory only	❌ **Bug**	After a button confirmation the agent still believed the booking was pending and refused to move it
Live booking status every turn + notes on button actions	✅ **Chosen**	The scheduler is the source of truth; the agent is told the current state

Chosen: booking_status_for_agent in every agent turn; note_for_agent whenever something changes outside the agent.

Code:

- `zovak/services/booking_state.py → booking_status_for_agent`
- `zovak/agents/memory.py → note_for_agent`

22.8 Call guardrails

Tried	Result	Why / evidence
30 s silence timer	❌ **Too slow / gameable**	Coughs and "uh" reset it
10 s warning + 10 s, real words only, noise-only turns end the call	✅ **Chosen**	Same rules on website, phone and WhatsApp calls; off-topic (3 strikes), too many changes, misuse report, per-caller call limit

Chosen: CallGuard bookkeeping fed by the sideband; the AI can report off-topic and misuse but can't book.

Code:

- `zovak/voice/call_guard.py → CallGuard`
- `zovak/voice/sideband.py → _watch_silence, _caller_said, _noise_only_turn, _end`
- `zovak/agents/receptionist.py → report_off_topic, report_misuse, end_call`

23. Change log

2026-10-30 — **Private demo for deployment: one backend setting, site key, access gate, test scripts**

One setting: serena-health/src/config/backend.ts holds ZOVAK's address (local 127.0.0.1:8000 by default; https://stapling-delete-stunner.ngrok-free.dev for Vercel), used by every website→ZOVAK call. Site key: through the tunnel ZOVAK used to refuse everything but the webhooks and forms; now the website's server may also reach chat, voice and staff routes if it sends X-Zovak-Site-Key matching SITE_API_KEY (_guard_public_host; empty key = closed, as before). Access gate: the chat/voice widget and the staff portal need the team code (DEMO_ACCESS_CODE, server-side only) and Amartya's note accepted; middleware answers 401 on /api/agent/* and /api/staff/* until then (HttpOnly cookie with a hash of the code, 7 days). Test scripts: a floating paper with short cost-saving scripts for Chat, Voice, Phone, WhatsApp (paused) and Staff, next to the widget and on /staff. Every card has a ✕ to hide it (never cancels anything).

2026-10-30 — **Streaming replies, relevant cards only, one time change after booking, "never mind"**

Streaming: website chat now streams Nofa's words as the AI writes them (stream_receptionist with Runner.run_streamed → /api/v1/dev/chat/stream → /api/agent/chat/stream), so text appears within about a second once she answers; tool calls still run first. Cards: after a reply a card shows only if it changed or the message is about the booking (time, doctor, email, status…); the last card is kept for the tab so it is there instantly after a refresh. Tapped time: Nofa treats it as chosen and asks only for what's missing. One change: a confirmed booking's time can be changed once by the patient (MAX_TIME_CHANGES, BookingProposal.time_changes, enforced in propose_move for every channel; staff delay moves don't count). The booked card has Change time with "You can change the time once after booking"; it opens Pick your new time, and a tap prepares the move to Confirm or Keep (Keep doesn't use it up). "No, forget it" / "never mind" / "not interested" clears doctor and time cards and drops an unconfirmed proposal or pending change, never a confirmed booking.

2026-10-30 — **Faster first reply, chat kept for the tab, no stray card after booking**

First reply: it needed three AI calls in a row (look up doctors, offer them, answer), about 15 s. The doctors and services are now given to Nofa up front (clinic_catalog), so it is two calls (~5 s measured in-process; the AI's own speed varies), and the AI library is loaded at backend start instead of during the first message (~2.5 s). Chat: the conversation id and messages are kept for the browser tab, so closing the panel or refreshing continues the same chat with its card. Card: confirming a booking clears the doctor and time picks, and after a booking, asking about free times or naming the doctor no longer opens a "Time selected… nothing is booked yet" card (doctor cards for a second appointment still work).

2026-10-30

"Change the mail" reopens the email box

Bug: typing "i would like to change the mail" in chat kept the review card, and Nofa pointed at an email field that wasn't there. Now plain code recognises a request to change the email ("mail" or "email", typos included; "did you get my email?" doesn't count): in chat and WhatsApp it clears the email on the waiting proposal (and the remembered one) so the email box shows again for the same appointment; on website calls the call's own on-screen email box reopens, now also for "mail".

2026-10-30

Google-backed scheduler kept wiping the typed email

Bug (seen live): after changing the time, the email box came back every time. The Google-backed scheduler's propose called the base propose without the email and then overwrote the result with the email Nofa passed (none), erasing the remembered one. Now it passes the email through and lets the base scheduler validate it and reuse the one already typed. A new test runs the change-the-time flow on the Google-backed scheduler itself, not only the demo one.

2026-10-30

Name and email remembered per conversation; no card flash; one reply per first message

Bugs: after a pause the email box came back on a time change (proposals expire after 5 minutes and took the email with them); once the email showed "already entered on screen" because Nofa passed a note instead of an address; the previous card flashed up before the new one; a first message from a page button was answered twice in development. Now the scheduler remembers the name and typed email for the whole conversation and fills them into any new proposal (Change email clears it, so it is asked again); anything that isn't a real address is ignored; Nofa is told the name and that the email is saved and to leave it out. The chat fetches the next card while the reply is written and shows both together. The "what's on screen" note is given only on the website (chat, web call), never on phone or WhatsApp. These rules live in shared ZOVAK code, so website chat, website calls, phone calls and WhatsApp all follow them. Email box hint: "Nothing is booked until you confirm."

2026-10-30

Change the time from the review card; name and email are kept

Bug: with the review card showing, "can I change the time" kept the review card, and saying "5 pm" made a new proposal that asked for the email again. Now a change-the-time request (typos like "chane" count; "check the time" doesn't) sets the waiting proposal aside, keeps the same doctor chosen and brings the time buttons back (DemoScheduler.reopen_times). A new proposal in the same conversation reuses the email already typed, and the name is passed to Nofa as already known, so the next step is straight to Review your appointment. On website calls an email typed on screen earlier in the call is trusted for the new time (EmailFormGate.arm(trusted_email)); Change email, or saying the email is wrong, still asks again.

2026-10-30

Naming a doctor directly shows their times; changing reopens the choice

Bug: "book with Dr. Farhan Karim for a skin rash" showed no doctor card and no times, while Nofa said "pick a time below". A choice only counted when picked from offered cards, and Nofa re-offering the same doctor wiped it. Now: naming exactly one clinic doctor chooses them even with no cards (and Nofa checking one doctor's times counts too); offering an already-chosen doctor keeps the choice; "change the time" / "another slot" brings the time buttons back and "a different doctor" starts the doctor choice again (read_choices, plain code before the AI). Nofa's live status now says what the website shows (WEBSITE SCREEN: doctor chosen, buttons showing or time picked), and she may only point at buttons that are really there.

2026-10-30

Time buttons always visible; Nofa doesn't list times on the website

Bug: Nofa typed out every free time, pushing the time buttons below the bottom of the chat, and the chat didn't scroll to new cards. Now any new or changed card (doctor cards, time buttons, booking card) is scrolled into view, and on the website Nofa never lists the times: she mentions at most the earliest one and points to the buttons, asking only for what's still missing.

2026-10-30

Time buttons collapse once a time is picked

Bug: after tapping a time the full list of buttons stayed under every reply. Now plain code recognises a tapped time in the patient's message (pick_tapped_time, only if it is one of the chosen doctor's free times) and ZOVAK remembers it; the card collapses to "Time selected · Fri, Oct 30 · 11:00 AM with Dr. …" with a Change time link that brings the buttons back. The picked time is forgotten when another doctor or kind of care is chosen, and when the booking is prepared.

2026-10-30

Free times as tap buttons on the website

After a doctor is chosen, the "Doctor selected" card lists that doctor's real free times for the next 3 days that have any (day tabs + time buttons): only starts inside their hours, not lunch, not booked, not past and not held by another patient (free_days in booking_state.py; none if the calendar can't be read, and typing still works). Chat: a tap sends "I'd like Friday, October 30 at 11:30 AM with Dr. …" as the patient's message. Website call: a tap goes to /api/v1/dev/voice/pick-slot, which accepts only one of those free times and runs it through the call exactly like speech (VoiceSideband.pick_slot), so Nofa answers aloud. A tap only proposes; confirming is unchanged. Chat now proposes as soon as doctor, time and name are known and leaves the email to the card's email box. Phone calls are unchanged (no screen).

2026-10-29 **Website: full-page light ribbons and light/dark themes (redesign branch)**

On redesign/animated: the light ribbons are one fixed background behind every section; they drift and tilt with scroll position and swell with scroll speed. A sun/moon switch in the header (and in the staff portal) toggles light and dark; the choice is remembered, dark is the default, and a small script in applies it before the page paints. All colours are CSS variables (globals.css), so the homepage, the chat/voice widget and the staff portal switch together; in dark the widget's accents become the homepage's aqua with dark text on bright buttons. No backend changes.

2026-10-29 **Website: animated "night shift" homepage on a separate branch**

The website as it was is committed and pushed to GitHub main (3c378ad). A new animated homepage lives on the branch redesign/animated (6cba4d0), so it can be dropped with git checkout main. Inspired by the reference videos: dark canvas with flowing light ribbons (canvas 2D, additive strands, paused off-screen, still under reduced motion), headlines that rise word by word with serif italic accents, a pinned scroll story of one booking (listen → right doctor → free time → confirm and goodbye), a scroll-lit statement of the one rule, channel cards with live mini demos, an interactive running-late planner that uses the same ripple rule as RecoveryPlanner.preview, an animated staff desk and a bright closing panel. Files: serena-health/src/components/night/*, src/app/page.tsx. No backend changes.

2026-10-29 **Staff assistant: a fresh memory per sign-in**

Bug: the staff assistant's memory outlived backend restarts and page reloads, while the chat on screen started empty and old delay plans were gone, so it mentioned plans that no longer existed ("Your 1-hour delay plan hasn't been applied"). Now each sign-in gets its own memory (a random id, never the sign-in token), the on-screen chat is kept in the tab while signed in and cleared on sign-in/sign-out, and the assistant answers only what was asked. Note: if the website's dev server stops rebuilding after edits, restart npm run dev.

2026-10-29 **Delays count from the right moment; staff chat scrolls and streams**

Bug: a delay always counted from "now" and only looked at the rest of today. At 10 PM Dr. Karim said he'd be 30 minutes late; his day had ended, so nobody was found, and the assistant offered to "apply" an empty plan. Now the delay counts from now while the doctor is working, from their start time before their day begins, and from the next day's start after it ends. Staff can also give a time ("30 minutes late for my 1:00"): the Running late page has "Late from: Now / A specific time", and the assistant passes late_from. Every preview says when it counts from; when nobody is affected it says why ("first patient is Chandni at 11:30; the delay ends at 11:30, in time for them") and there is nothing to send. The assistant re-checks on every report instead of reusing an old answer from earlier in the chat. The staff assistant chat now keeps the newest message in view and writes replies out word by word with a typing indicator.

2026-10-29 **Doctors can see any day in their schedule**

The staff Schedule tab had only Today and Tomorrow, so a booking for next week was saved but invisible in the doctor's login. Now there are ‹ › buttons to step a day at a time and a date picker for any day. Bookings from website chat, website voice and phone all land in the same booking store the portal reads (refreshed every 5 s); doctors have no calendars of their own.

2026-10-29 **Warm automatic goodbyes, matching doctors only, compact call list, new greeting**

Goodbyes: plain code now checks everything the caller says, not only what the voice model passes on (it often answered "bye" itself, so the call stayed open). A clear goodbye ("thanks, bye", "I have to go", "hang up"; not "okay" alone, not "bye, but first…") ends the call with a personal message ("You're all set with Dr. Rahman tomorrow at 11:30 AM. Thanks for calling Brightside Family Clinic. Take care, goodbye!"). If a booking or change is half-done, Nofa asks once ("Before you go, your appointment isn't booked yet…"); a second goodbye ends it with nothing booked or changed. Doctors: offer_doctors refuses doctors from more than one specialty, so a list of every doctor is never shown or read out; asked "which doctors do you have?", Nofa describes the kinds of care and asks about the problem first. Website calls show the matching doctors as a slim scrolling list so the transcript stays visible. New chat greeting and buttons.

2026-10-29 **Doctor cards go away once a doctor is picked**

Bug: the doctor cards stayed under every message until the whole booking was prepared, so the patient kept seeing "Choose" after choosing. Now plain code (not the AI) checks each chat message and spoken turn: if it names exactly one offered doctor (tapping Choose, typing or saying the name; close speech spellings count; "not Rahman" or naming both doesn't), the cards are replaced by a "Doctor selected" card that says what happens next. Nofa checking one offered doctor's times also counts as a pick, and she doesn't re-offer the list unless asked. Booked/cancelled cards are no longer hidden by new doctor cards.

2026-10-29 **Manual test plan**

New docs/ZOVAK_Test_Plan.pdf (built by docs/build_test_plan.py): numbered hands-on test cases with Pass/Fail boxes for website chat, website voice, phone calls and the SMS form, the staff portal (sign-in, running late, patient choices), the staff assistant, an end-to-end story and security checks. WhatsApp tests are paused while the Meta account is under review.

2026-10-29 **WhatsApp typing indicator shown once**

After Meta restricted the business account (error 131031, automated account-integrity review), the typing indicator is sent once per message instead of refreshed every 20 s, keeping API traffic human-paced. No Meta API calls were ever made by the tests.

2026-10-28 **Delay recovery engine + staff portal (phases 2 and 3)**

A doctor's delay ripples forward only until free time absorbs it; each affected patient gets options (same time with another free doctor, the shifted time, the doctor's next free times after they arrive, cancel). Staff preview first; sending applies shifted times latest-first and notifies patients with a one-time /update link. New /staff portal: sign-in, schedule per doctor, Running late with preview, patient updates, staff assistant. Demo bookings are now created around the current time, and the two GPs are staggered.

2026-10-28 **Doctors and staff (phase 1 of schedule recovery)**

Four demo doctors (General Practice ×2, Pediatrics, Dermatology) with their own services, hours and lunch breaks; two officials; demo bookings for today/tomorrow loaded at startup. Every booking belongs to a doctor; availability, holds and overlaps are per doctor. Nofa first asks what kind of doctor the patient needs, then offers matching doctors (cards on the website, names on calls) or says the clinic doesn't offer it.

2026-10-28 **WhatsApp calls share the person's WhatsApp chat conversation**

Calls now use the same conversation as that number's chat (wa-), so a booking made in chat is known on a call and vice versa, with shared agent memory. Safe because Meta verifies WhatsApp numbers (unlike phone caller ID). Phone and website still have their own conversations until customer tracking is built.

2026-10-28 **WhatsApp calls work through the audio bridge; decision record added**

A live WhatsApp call to the test number was answered by Nofa through the bridge. Added a decision record explaining every route tried, why each failed or was chosen, and where the code is.

2026-10-28 **WhatsApp calls through ZOVAK's audio bridge**

Diagnostics confirmed Meta and GPT-Live are both ICE-lite (a=ice-lite in both SDPs), so neither starts the connection and a direct call never gets audio. New AudioBridge (aiortc) answers Meta's offer and calls GPT-Live like a browser, relaying audio both ways at real time (silence until audio arrives, one fixed mono 48 kHz format, backlog capped). Verified locally with two real WebRTC peers: tones flowed both ways.

2026-10-28 **WhatsApp call experiment: no audio; SDP diagnostics**

The first WhatsApp test call was accepted but no audio flowed and Meta ended it as FAILED after ~21 s. ZOVAK now logs the connection-relevant SDP lines of Meta's offer and GPT-Live's answer (ice-lite, setup role, candidates) to confirm why.

2026-10-28 **WhatsApp working: subscribe the account to the app; typing indicator**

Messages showed in Meta's 'Check test webhooks' but never reached ZOVAK: the test flow had subscribed the WhatsApp account only to Meta's test viewer app. POST /{WABA_ID}/subscribed_apps with the app's token fixed it. Nofa now marks messages read and shows 'typing…' (refreshed every 20 s) while the agent works.

2026-10-28 **WhatsApp chatbot + WhatsApp calling experiment**

Nofa answers WhatsApp messages (Meta Cloud API, free test number): same agent, email typed in chat, tap buttons for Confirm booking / Confirm move / Cancel / Keep, signed webhooks, rate and change limits. Experimental WhatsApp calls: Meta's WebRTC offer is handed to GPT-Live and its answer returned to Meta, with the phone call brain attached.

2026-10-28 **Correction: Sweden SMS was not delivered**

Telnyx accepted the Sweden test text (queued) but the Swedish carrier rejected it (40008 Undeliverable), most likely the unregistered NofaClinic sender. Real SMS through Telnyx is unreliable without a registered sender; use ntfy for testing.

2026-10-28 **SMS router: test inbox, ntfy push, Telnyx by country**

Every text goes through SmsRouter.send(): always to the test inbox; to the ntfy app on your phone when TEST_SMS_NTFY_TOPIC is set (free testing); and to a real provider chosen by country code. Telnyx is wired for non-Bangladeshi numbers; +880 is skipped because Telnyx requires a registered sender there. A Bangladeshi gateway plugs in later as one adapter.

2026-10-27 **Noise guardrail; how calls connect, in detail**

Coughs, sniffs, 'uh'/'hmm' and transcription tags like [cough] no longer reset the silence timer. A turn with only noise isn't sent to the AI; right after the 10 s warning (someone pretending to answer), or on the 3rd noise-only turn, Nofa ends the call politely. The documentation now walks through how WebRTC and SIP calls connect to GPT-Live, with code.

2026-10-27 **First successful phone booking through Telnyx**

Linphone direct to OpenAI connected but no audio ever flowed (no packets, nothing billed). A Telnyx US number with an FQDN SIP connection to OpenAI (TLS + SRTP) worked end to end: Nofa answered, the booking was prepared, 'confirm' sent the form, and the form booked it.

2026-10-26 **Phone call diagnostics**

First Linphone test: the call was accepted but no audio was heard, and the logs couldn't show why. ZOVAK now prints its own timestamped log lines (caller said / Nofa replies / caller audio arriving / Nofa speaking), logs whether the call audio is encrypted (SRTP) or plain, greets phone callers first, and only starts the silence timer once caller audio has arrived. If no caller audio arrives within 45 s it ends the call as 'no_audio' and says so in the log.

2026-10-26	**Phone calls (Linphone/SIP), SMS booking form, safer natural yes, call guardrails** Phone calls reach ZOVAK through GPT-Live SIP (tested free with Linphone). On a call, saying 'confirm' texts a one-time 2-minute form; the form is what books (edit name, add email, pick another time, or cancel). Spoken yes accepts natural wording but never a reply that could be the start of 'yes, but…' (action word required, hedge words block it, and ZOVAK waits for silence first). Calls end after 10 s silence + a 10 s warning, when the caller says that's all, or when someone keeps booking/moving/cancelling. Pending proposals now hold their slot.
2026-10-24	**Agent always knows the real booking status** Bug: after confirming with the on-screen button, the agent still believed the booking was pending (its memory only had the 'PENDING' tool result) and refused to move it. Fix: every agent turn now gets the live booking status from the scheduler, and every button or strict spoken confirmation adds a note to the agent's memory.
2026-10-24	**Widget polish, call timer, card/transcript overlap fix, this documentation** Widget font changed to Figtree; chat bubbles restyled; the voice panel shows a compact header with a live call timer during a call; the booking card can no longer be squeezed over the transcript. Documentation PDF added and rebuilt after every change.
2026-10-24	**Call ends on silence or off-topic requests** CallGuard counts caller silence (30 s, 2 min while the email box is open) and off-topic strikes (3). Nofa checks in, says goodbye, and the website hangs up.
2026-10-24	**Booking card, move, cancel, reminders, today's date** Shared booking-state builder for chat and voice; Confirm booking button; same-conversation move and cancel with Google Calendar update/delete; 1-hour Resend reminders; the agent and GPT-Live are told today's date so Nofa never asks for the year.
2026-10-23	**Typed email box for voice calls** Callers type their email on screen instead of spelling it aloud; typing never books.
2026-10-23	**Website connected to ZOVAK** The Nofa website's chat and voice call talk to the local ZOVAK backend through Next.js routes that forward requests server-side.